R Programming Language

Luka Jokic

January 2022

“The only way to learn a new programming language is by writing programs in it.”

Dennis Ritchie

Section 1

Introduction to R

Welcome to R! R is a programming language and software environment for statistical analysis, graphics representation and reporting. In this section we will master the basics of this widely used open source language, including data types, matrices, lists, and data frames. We will learn to install packages, write functions, use loops, relational and logical operators.

Subsection 1

Installing R and R Studio

Install R

To download R, follow the following steps:

1. Go to *https://cran.r-project.org*.
2. Click on *Download R for Windows*.
3. Click on *base*.
4. Click on *Download R 4.1.2*[1] *for Windows*.

When you download the *exe* file, run it and complete the installation (you can finish the installation with default instructions).

[1] This version might be different when you install R.

Install R Studio

To download R Studio, follow the steps below:

1. Go to *https://www.rstudio.com/*.
2. Click on *Download* tab.
3. Scroll down and click on *Download* button bellow RStudio Desktop option.
4. Click on *Download RStudio for Windows*.

After you have your R Studio downloaded, finish the installation following the basic default steps.

Subsection 2

Basics in R

Arithmetic operators

All basic **arithmetic operators** are available in R, such as:

Arithmetic operation	**Command**
Addition	+
Subtraction	-
Multiplication	*
Division	/
Exponentiation	^
Modulo	%%

```r
# An addition
5 + 2
```

```
## [1] 7
```

```r
# A subtraction
5 - 2
```

```
## [1] 3
```

```r
# A multiplication
3 * 6
```

```
## [1] 18
```

```r
# A division
(5 + 2) / 2
```

```
## [1] 3.5
```

```r
# Exponentiation
3 ^ 2
```

```
## [1] 9
```

```r
# Modulo
27 %% 5
```

```
## [1] 2
```

Variable assignment

A basic concept in (statistical) programming is called a **variable**.

A variable allows you to store a value or an object in R. You can then later use this variable's name to easily access the value or the object that is stored within this variable.

Let's assume you have information about total revenue and total expenses of one company. You can assign these values to separate variables `tot_rev` and `tot_exp` with the command <- (alt + minus).

```
tot_rev <- 200
tot_exp <- 100
```

Profit of this company is easily calculated as the difference between total revenues and total expenses. Arithmetic operators work with numeric variables in R. Again, new variable `profit` can be used to store obtained result.

```
profit <- tot_rev - tot_exp
profit
```

```
## [1] 100
```

Basic data types in R

There are various data types in R. Some of the most commonly used types are:

- **Numeric** - decimal values are called numerics. Natural numbers are called integers. Integers are also numerics.
- **Logical** - boolean values (TRUE or FALSE) are called logical.
- **Character** - text (or string) values are called characters. Quotation marks indicate that some text is a character.

```r
# Example
num_value <- 21
char_value <- "student"
logical_value <- FALSE

# Check class
class(num_value)
```

```
## [1] "numeric"
```

```r
class(char_value)
```

```
## [1] "character"
```

```r
class(logical_value)
```

```
## [1] "logical"
```

```r
# Integer vs. double
typeof(42)
```

```
## [1] "double"
```

```r
typeof(42.0)
```

```
## [1] "double"
```

```r
typeof(42L)
```

```
## [1] "integer"
```

Vectors

A **vector** is created with the function `c()` in R. Elements of one vector are separated by a comma inside `c()` function.

```
prices <- c(120, 220, 50)
prices
```

```
## [1] 120 220  50
```

You can assign names to the elements of a vector with the `names()` function.

```
names(prices) <- c("Apple", "Microsoft", "Intel")
prices
```

```
##     Apple Microsoft     Intel
##       120       220        50
```

All arithmetic operators can be used when working with vectors. For example, we can sum two vectors using the + operator. One by one element is summed and a new vector is created.

```
x <- c(2, 5, 3)
y <- c(7, 1, 5)
x + y
```

```
## [1] 9 6 8
```

In order to calculate the sum of all elements of a vector, we use `sum()` function.

```
sum(x)
```

```
## [1] 10
```

If we want to select certain elements of a vector, we use square brackets where we point out numeric positions of these elements. Note that the first element in a vector has index 1. Let's return to our stock prices example. To select first element of this vector, we simply write `prices[1]`. Another way to extract the vector element is by using the names of the elements (e.g. `prices["Apple"]`).

```
prices[1]
```

```
## Apple
##   120
```

```
prices["Apple"]
```

```
## Apple
##   120
```

You can select multiple elements by writing the vector of elements' positions that should be selected inside square brackets.

```
prices[c(2, 3)]
```

```
## Microsoft     Intel
##       220        50
```

Also, if you want to select consecutive elements, instead `prices[c(2, 3)]`, you can write `prices[2:3]`.

```
prices[2:3]
```

```
## Microsoft     Intel
##       220        50
```

Relational operators

Relational operators are operators which help us to see how one R object relates to another. If we want to see whether two objects are equal, we use double equal sign ==. The opposite of equality operator is inequality operator !=.

```
# Examples
5 == 5
```

```
## [1] TRUE
```

```
"R" != "Python"
```

```
## [1] TRUE
```

```
TRUE == FALSE
```

```
## [1] FALSE
```

To check if an R object is less than or greater than another R object, we use < and > signs. Note that if we use these signs to compare two strings, R uses alphabetical order when sorting character strings. As for logical values, have in mind that `TRUE` corresponds to 1, and `FALSE` to 0. In order to check whether one object is greater (less) or equal to another object, we use >= (<=) sign.

```
# Examples
8 > 3
```

```
## [1] TRUE
```

```
"Belgrade" > "City"
```

```
## [1] FALSE
```

```
TRUE > FALSE
```

```
## [1] TRUE
```

```
4 <= 4
```

```
## [1] TRUE
```

Relational operators can handle vectors as well. For example, we can check whether monthly returns are greater than some predefined value, or compare two vectors of returns. Comparison is done for every element of a vector.

```
monthly_returns <- c(11, 9, 6, 12, 7)
monthly_returns > 10
```

```
## [1]  TRUE FALSE FALSE  TRUE FALSE
```

```
c(5, 3) <= c(2, 8)
```

```
## [1] FALSE  TRUE
```

Recall that we use square brackets to select certain elements of a vector (Chapter Vectors). Now, we can use relational operators to select only elements that satisfy some condition.

```
# Example
monthly_returns[monthly_returns > 10]
```

```
## [1] 11 12
```

Logical operators

If we want to change or combine results of previous comparisons made by using relational operators, we use **logical operators**, i.e. AND (&), OR (|) and NOT (!) operators.

When comparing two logical values, **AND** operator returns TRUE only if both logical values are TRUE. If we want to check whether some variable is both greater than some value and less than some other predefined value, we use AND operator.

```
# AND operator - examples
TRUE & TRUE
```

```
## [1] TRUE
```

```
FALSE & TRUE
```

```
## [1] FALSE
```

```
x <- 7
x > 3 & x < 10
```

```
## [1] TRUE
```

The difference between **OR** and AND operator is that OR returns TRUE if at least one logical value equals to TRUE.

```
# OR operator - examples
FALSE | TRUE
```

```
## [1] TRUE
```

```
FALSE | FALSE
```

```
## [1] FALSE
```

```
x <- 7
x > 3 | x < 6
```

```
## [1] TRUE
```

The **NOT** operator negates the logical value.

```
# NOT operator - examples
!TRUE
```

```
## [1] FALSE
```

```
x <- 7
!(x < 3)
```

```
## [1] TRUE
```

Similar to relational operators, logical operators perform operations on vectors element-wise (one by one element). However, in order to compare only first element of two vectors, we use double operator sign, i.e. && and ||.

```
# Logical operators on vectors - examples
c(TRUE, FALSE) & c(TRUE, TRUE)
```

```
## [1]  TRUE FALSE
```

```
c(TRUE, FALSE) && c(TRUE, TRUE)
```

```
## [1] TRUE
```

Factors

Factor is another data type used for dealing with categorical variables. Categorical variables consist of limited number of categories. It is very important to recognize and define categorical variables, as they are treated differently in statistical models you will use. Examples of categorical variables are race, sex, age group, and educational level.

The factor variable is defined by the `factor()` function.

```
#M stands for male, F for female
sex <- c("M", "F", "F", "M", "F", "F")
sex_fct <- factor(sex)
```

Sometimes you will want to change the names of levels of a categorical variable. To change names, you use the `levels()` function. Consider again example with the sex variable, and assume you want to change factor levels to `'Male'` and `'Female'`.

```
sex_fct
```

```
## [1] M F F M F F
## Levels: F M
```

```
levels(sex_fct) <- c("Female", "Male")
```

This was an example of a nominal categorical variable where you can not order the categories and therefore cannot compare different categories.

```
sex_fct[1] > sex_fct[2]
```

```
## Warning in Ops.factor(sex_fct[1], sex_fct[2]): '>' not meaningful for factors
```

```
## [1] NA
```

However, some categorical variables are ordinal, so you need to define the levels of a variable.

```
education <- c("college", "primary", "high_school", "primary", "college")
education_fct <- factor(
  education, ordered = TRUE,
  levels = c("primary", "high_school", "college")
)

education_fct[1] > education_fct[2]
```

```
## [1] TRUE
```

Subsection 3

Functions and packages

Functions

One of the most powerful tools in R are **functions**. We have already used some functions such as `c()` and `sum()`. To call a function you simply write so called function arguments inside parenthesis and the function gives an output.

There can be one or more arguments inside a function. Again, you can assign results of a function in a variable.

Let's explore the `sd()` function. If you are not sure what some function does or how it should be used, take a look in the R documentation using the following commands:

```
help(sd)
?sd
```

sd {stats} R Documentation

Standard Deviation

Description

This function computes the standard deviation of the values in `x`. If `na.rm` is `TRUE` then missing values are removed before computation proceeds.

Usage

```
sd(x, na.rm = FALSE)
```

Arguments

`x`	a numeric vector or an R object but not a `factor` coercible to numeric by `as.double(x)`.
`na.rm`	logical. Should missing values be removed?

Not surprisingly, `sd()` function computes the standard deviation. In the *Usage* section, we see that the `sd()` function takes two arguments.

```
sd(x, na.rm = FALSE)
```

The first argument (i.e. `x`) should be a numeric vector. When you call an R function, R has to match your input values to the function's arguments.

```
values <- c(2, 5, 3, 7)
sd(values)
```

```
## [1] 2.217356
```

R matches the values to the `x` argument by position. Therefore, R knows that `values` variable is the first argument. However, you can match arguments by name by specifically saying that we want `x` argument to be `values` variable.

```
sd(x = values)
```

The second argument is `na.rm`. It is a logical value, indicating whether missing values[2] should be removed or not.

```
values <- c(2, 5, NA, 7)
sd(values)
```

```
## [1] NA
```

[2] Missing values are denoted as `NA` in R.

Since the default value of the `na.rm` argument is `FALSE`, missing values were not removed, so the result is just `NA`. Hence, for the case when the values contain missing values, we have to specify the `na.rm` argument and set it to `TRUE`. The function will then remove missing values before calculating the standard deviation.

```
sd(values, TRUE) #by position
```

```
## [1] 2.516611
```

```
sd(x = values, na.rm = TRUE) #by name
```

```
## [1] 2.516611
```

```
sd(values, na.rm = TRUE) #mix
```

```
## [1] 2.516611
```

If we do not define the `na.rm` argument, the default value of this argument is taken. However, we could not leave `x` argument unspecified as `x` argument does not have a default value.

Writing functions

If you need to solve a particular problem, but you do not know already built function that can serve your need, you may want to write your own function. You use the `function` construction for this.

```
function(arg1, arg2) { body }
```

Previously, you saw that function works such that you provide input values and the function generates the output. The first step is to think of the body of the function, i.e. what is the purpose of the function. Let's assume we want to create a function that calculates the squared difference. Secondly, we need to define function arguments.

```r
# Example
my_fun <- function(x, y) {
  (x - y) ^ 2
}
my_fun(5, 2)
```

```
## [1] 9
```

The last expression evaluated inside the function becomes the return value. You can also specify the return value, by using the `return` statement.

```
my_fun <- function(x, y) {
  z <- (x - y) ^ 2
  return(z)
}
my_fun(5, 2)
```

```
## [1] 9
```

However, if we do not specify both arguments, function will generate an error because none of the arguments have their default values defined. In the following example, the default value of the second argument is set to 0. Therefore, only first argument is required.

```
my_fun <- function(x, y = 0) {
  (x - y) ^ 2
}
my_fun(5)
```

```
## [1] 25
```

Packages

All built-in functions are part of so-called **R packages**. R packages include lot of different functions, data, documentation and tests. In order to use a package and all the content related to it, you need to install it firstly. Some packages are automatically installed when you install R. The `base` package is an example. It contains a lot of basic R functions, such as `sum()` and `max()` functions.

However, many useful packages are not automatically installed. You can install it by using the function `install.packages()`, which is the part of the `utils` package. This function downloads the package file from the CRAN[3] and installs it on your system. Note that you need to type the name of the package inside quotation marks.

```
install.packages("tidyverse")
```

[3] Comprehensive R Archive Network

After you install the package, you have to load it into current R session to be able to use it. Again, some packages are automatically loaded when R is started, among which is the `base` package. That is why you can use a lot of functions without previously installing or loading some packages. To load the package, we use the `library()` command. You can then use all the content from the loaded package.

```
library(tidyverse)
```

If you package is installed, you can also call certain functions from the package without loading it with the `package::function()` command.

```
base::max(c(4, 5))
```

```
## [1] 5
```

Note that you can use `require()` function as well to load a package. The only difference between `require()` and `library()` is that `require()` does not generate an error if you try to load the package that is not yet installed, but warning instead. This is helpful when you are loading packages inside functions to avoid getting errors.

Subsection 4

Matrices, Data frames and Lists

Matrices

A **matrix** is a two-dimensional group of same data type elements arranged into rows and columns. It is constructed with the `matrix()` command.

This function contains several important arguments. The first argument corresponds to data, i.e. elements of the matrix. With the `byrow` argument you indicate whether the matrix will be filled by rows or columns. It takes logical values, so if we want the matrix to be filled by the columns, we type `byrow = FALSE`. The argument `nrow` (`ncol`) indicates how many rows (columns) the matrix should have.

```
matrix(c(2, 5, 1, 3), byrow = TRUE, nrow = 2)
```

```
##      [,1] [,2]
## [1,]    2    5
## [2,]    1    3
```

```
matrix(c(2, 5, 1, 3), byrow = FALSE, nrow = 2)
```

```
##      [,1] [,2]
## [1,]    2    1
## [2,]    5    3
```

You can assign names to the rows and the columns of the matrix using the `rownames()` and `colnames()` functions. However, the `matrix()` function has one additional argument `dimnames` where you define a list (Chapter Lists) of row and column names.

```
mm <- matrix(c(2, 5, 1, 3, 4, 3), byrow = TRUE, nrow = 2)
rownames(mm) <- c("A", "B")
colnames(mm) <- c("A", "B", "D")
mm
```

```
##   A B D
## A 2 5 1
## B 3 4 3
```

```
matrix(
  c(2, 5, 1, 3, 4, 3),
  byrow = TRUE,
  nrow = 2,
  dimnames = list(c("A", "B"), c("A", "B", "D"))
)
```

```
##   A B D
## A 2 5 1
## B 3 4 3
```

Again, you can use square brackets [] to select elements from the matrix. Since matrices have two dimensions, you should use comma to separate rows and columns you want to select. If you want to select all elements of one row or column, you leave blank space after or before the comma.

```
mm[1, 3] #first row, third column
```

```
## [1] 1
```

```
mm[1:2, 2:3] #first two rows, second and third column
```

```
##   B D
## A 5 1
## B 4 3
```

```
mm[2,] #all elements from the second row
```

```
## A B D
## 3 4 3
```

```
mm[,3] #all elements from the third column
```

```
## A B
## 1 3
```

You can add one or multiple columns (rows) to a matrix using the `cbind()` (`rbind()`) function, which merges matrices and/or vectors by column (row).

```
matrix_1 <- matrix(1:9, byrow = TRUE, nrow = 3)
matrix_2 <- matrix(1:6, byrow = FALSE, ncol = 2)
```

```
cbind(matrix_1, matrix_2)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    2    3    1    4
## [2,]    4    5    6    2    5
## [3,]    7    8    9    3    6
```

```
rbind(matrix_1, 10:12)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
## [4,]   10   11   12
```

All arithmetic operations can be used when working with matrices. Note that if you want to do matrix multiplication, you use %*% command. `rowSums()` and `colSums()` are two useful functions used for computing the totals of each row and column of a matrix respectively. To transpose a matrix, you use the `t()` function.

```
matrix_1 * 3
```

```
##      [,1] [,2] [,3]
## [1,]    3    6    9
## [2,]   12   15   18
## [3,]   21   24   27
```

```
matrix_1 %*% matrix_2 #matrix multiplication
```

```
##      [,1] [,2]
## [1,]   14   32
## [2,]   32   77
## [3,]   50  122
```

```
rowSums(matrix_1)
```

```
## [1]  6 15 24
```

```
colSums(matrix_2)
```

```
## [1]  6 15
```

```
t(matrix_1)
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

Data Frames

You will often have to work with data sets that contain different data type columns. A **data frame** has the variables of a different type as columns and the observations as rows. There are a lot of built-in data frames in R packages. One of the most common example is `mtcars` data set which is the part of `datasets` package automatically installed and loaded in R.

```
help(mtcars)
```

mtcars {datasets} R Documentation

Motor Trend Car Road Tests

Description

The data was extracted from the 1974 *Motor Trend* US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973–74 models).

Usage

```
mtcars
```

Format

A data frame with 32 observations on 11 (numeric) variables.

```
 [, 1] mpg  Miles/(US) gallon
 [, 2] cyl  Number of cylinders
 [, 3] disp Displacement (cu.in.)
 [, 4] hp   Gross horsepower
 [, 5] drat Rear axle ratio
 [, 6] wt   Weight (1000 lbs)
 [, 7] qsec 1/4 mile time
 [, 8] vs   Engine (0 = V-shaped, 1 = straight)
 [, 9] am   Transmission (0 = automatic, 1 = manual)
 [,10] gear Number of forward gears
 [,11] carb Number of carburetors
```

This data frame is not large. However, you will often work with huge data frames with thousands of observations. Therefore, it is useful to check only first few observations using the `head()` function. Similarly, the `tail()` function prints out the last observations.

```
head(mtcars)
```

```
##                    mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```
tail(mtcars)
```

```
##                 mpg cyl  disp  hp drat    wt qsec vs am gear carb
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.7  0  1    5    2
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.9  1  1    5    2
## Ford Pantera L 15.8   8 351.0 264 4.22 3.170 14.5  0  1    5    4
## Ferrari Dino   19.7   6 145.0 175 3.62 2.770 15.5  0  1    5    6
## Maserati Bora  15.0   8 301.0 335 3.54 3.570 14.6  0  1    5    8
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.6  1  1    4    2
```

Another useful function that is used to get a quick overview of the data is the `str()` function. It tells you how many observations and variables are there, data types and names of each variable, and first observations. This function gives a great insight in you data set.

```r
str(mtcars)
```

```
## 'data.frame':    32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

You construct a data frame with the `data.frame()` command. Arguments of this function are variables of different data types that become columns of your data frame. Note that columns must have the same number of observations, i.e. they should have the same length. Let's construct a data frame with three columns of different type.

```
df <- data.frame(
  name = c("Client A", "Client B", "Client C"),
  revenue = c(230.5, 111.2, 460.9),
  bankrupt = c(FALSE, TRUE, FALSE)
)
```

```
df
```

```
##       name revenue bankrupt
## 1 Client A   230.5    FALSE
## 2 Client B   111.2     TRUE
## 3 Client C   460.9    FALSE
```

```
str(df)
```

```
## 'data.frame':    3 obs. of  3 variables:
##  $ name    : chr  "Client A" "Client B" "Client C"
##  $ revenue : num  230 111 461
##  $ bankrupt: logi  FALSE TRUE FALSE
```

Similar to matrices, you choose elements from a data frame using square brackets `[]`. You can also use variable names to select specific columns. Obtained output is data frame as well. If you want to extract the entire column or specific element in form of vector you use double square brackets `[[]]`. Similarly, you can select the entire column by using the $ sign.

```
df[2, 3]
```

```
## [1] TRUE
```

```
df[1:2, 2:3]
```

```
##   revenue bankrupt
## 1   230.5    FALSE
## 2   111.2     TRUE
```

```
df[1,]
```

```
##       name revenue bankrupt
## 1 Client A   230.5    FALSE
```

```
df[1:2, "bankrupt"]
```

```
## [1] FALSE  TRUE
```

```
df[[2]]
```

```
## [1] 230.5 111.2 460.9
```

```
df$name
```

```
## [1] "Client A" "Client B" "Client C"
```

Lists

Lists are collection of different objects, such as matrices, vectors, data frames or even other lists. These objects differ in characteristic, length, and are not necessarily related to each other in any way. Lists are created with the function `list()`.

```
vector_list <- 1:5
matrix_list <- matrix(1:9, byrow = T, nrow = 3)
list(vector_list, matrix_list, df)
```

```
## [[1]]
## [1] 1 2 3 4 5
##
## [[2]]
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
##
## [[3]]
##       name revenue bankrupt
## 1 Client A   230.5    FALSE
## 2 Client B   111.2     TRUE
## 3 Client C   460.9    FALSE
```

You can name components of the list either directly inside the `list()` function or by using the `names()` function.

```
list(my_vector = vector_list, my_matrix = matrix_list, my_dataframe = df)
```

```
## $my_vector
## [1] 1 2 3 4 5
##
## $my_matrix
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
##
## $my_dataframe
##       name revenue bankrupt
## 1 Client A   230.5    FALSE
## 2 Client B   111.2     TRUE
## 3 Client C   460.9    FALSE
```

```r
list_example <- list(vector_list, matrix_list, df)
names(list_example) <- c("my_vector", "my_matrix", "my_dataframe")
list_example
```

```
## $my_vector
## [1] 1 2 3 4 5
##
## $my_matrix
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
## [3,]    7    8    9
##
## $my_dataframe
##       name revenue bankrupt
## 1 Client A   230.5    FALSE
## 2 Client B   111.2     TRUE
## 3 Client C   460.9    FALSE
```

Sometimes you will need to select component from the list. One way to do it is by defining the position of that component in the list using double square brackets `[[]]`. In the previous example we had three components. If we need to extract the vector from our list we can just write `list_example[[1]]`. We can also refer to the names of the component with the `[[]]` or `$`.

```
list_example[[1]]
```

```
## [1] 1 2 3 4 5
```

```
list_example[["my_vector"]]
```

```
## [1] 1 2 3 4 5
```

```
list_example$my_vector
```

```
## [1] 1 2 3 4 5
```

Beside that, you often need specific elements from list components. Let's see how we can find only first element of our vector from the `list_example` list.

```
list_example[[1]][1]
```

```
## [1] 1
```

```
list_example[["my_vector"]][1]
```

```
## [1] 1
```

Section 2

Data Analytics

In this section we focus on manipulation of data we work with, such as importing data, selecting, adding and renaming columns, filtering rows based on defined conditions, arranging the table by one or more columns, etc. We learn to tidy data, group data by appropriate column(s) and summarize it with some of the aggregate functions. Also, we combine different datasets using JOIN functions. Finally, we save obtained results in different formats.

We will primarily focus on a powerful set of tools known as the **tidyverse**. The tidyverse is a collection of R packages designed for data science. You will learn the processes of data manipulation using different tools from this package.

Subsection 1

Importing Data

This subsection focuses on importing flat files and excel files into R and converting it into R data frame. However, R includes functions for importing different type of data, such as data from databases (SQL) and data from other statistical software like SAS, SPSS, Stata. Also, we will learn functions for importing financial data.

Importing flat files with the `readr` package

`readr` contains functions used for importing data. It is a part of the `tidyverse` group of packages which is a collection of data science tools for transforming and visualizing data. The `tidyverse` is a very powerful and popular set of tools for exploring data which we will use in later chapters as well.

Apart from the `readr` package, the `utils` package provides functions which could also be used for importing data.

However, compared to `utils` package, which is more verbose and multiple times slower, `readr` package is fast and very easy to use. `readr` is not installed and loaded by default, so before using this package, you have to install it first.

Let's start with flat files. Flat files are simple text files that can have values that are either comma separated, tab separated, or separated by some other separator (e.g. forward slash, semicolon).

Have a look at a tab separated data file example. It consists of Facebook stock information (only first few rows are shown).

```
##  [1] "Index\tFB.Open\tFB.High\tFB.Low\tFB.Close\tFB.Volume\tFB.Adjusted"
##  [2] "2013-01-02\t27.440001\t28.18\t27.42\t28\t69846400\t28"
##  [3] "2013-01-03\t27.879999\t28.469999\t27.59\t27.77\t63140600\t27.77"
##  [4] "2013-01-04\t28.01\t28.93\t27.83\t28.76\t72715400\t28.76"
##  [5] "2013-01-07\t28.690001\t29.790001\t28.65\t29.42\t83781800\t29.42"
##  [6] "2013-01-08\t29.51\t29.6\t28.860001\t29.059999\t45871300\t29.059999"
##  [7] "2013-01-09\t29.67\t30.6\t29.49\t30.59\t104787700\t30.59"
##  [8] "2013-01-10\t30.6\t31.450001\t30.280001\t31.299999\t95316400\t31.299999"
##  [9] "2013-01-11\t31.280001\t31.959999\t31.1\t31.719999\t89598000\t31.719999"
## [10] "2013-01-14\t32.080002\t32.209999\t30.620001\t30.950001\t98892800\t30.950001"
```

It is saved as a txt file on my computer. The first line gives the names of different columns. Apart from the first line, each other line is a record and the columns are separated by tab separator. For example, the Facebook stock closing price was 28 on January 2, 2013.

There is a `read_tsv()` function from the `readr` package that provides functionality to import flat files into R as a tibble.

```
library(readr)
read_tsv("../data/FB.txt")
```

```
## # A tibble: 5 x 7
##   Index      FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
##   <date>       <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
## 1 2013-01-02    27.4    28.2   27.4     28    69846400        28
## 2 2013-01-03    27.9    28.5   27.6     27.8  63140600        27.8
## 3 2013-01-04    28.0    28.9   27.8     28.8  72715400        28.8
## 4 2013-01-07    28.7    29.8   28.6     29.4  83781800        29.4
## 5 2013-01-08    29.5    29.6   28.9     29.1  45871300        29.1
```

When using `readr` functions, obtained output is a tibble, which is a specialized version of data frame.

```
tibble(
  time = as.Date('2020-01-01') + 0:4,
  stock_A = rnorm(5, 0, 1),
  stock_B = rnorm(5, 0, 2),
  stock_C = rnorm(5, 0, 4)
)
```

```
## # A tibble: 5 x 4
##   time       stock_A stock_B stock_C
##   <date>       <dbl>   <dbl>   <dbl>
## 1 2020-01-01   2.24    0.468   0.820
## 2 2020-01-02  -1.64   -4.20   -0.834
## 3 2020-01-03  -2.31    0.771   0.812
## 4 2020-01-04   0.652   0.705   3.63
## 5 2020-01-05   1.02   -2.47   -1.23
```

It behaves like a normal data frame, but you can do additional stuff with it.

Another common format of flat files is the comma separated file. Assume that we have the same data but now all values are separated by comma, and stored into a .csv file.

```
##  [1] "Index,FB.Open,FB.High,FB.Low,FB.Close,FB.Volume,FB.Adjusted"
##  [2] "2013-01-02,27.440001,28.18,27.42,28,69846400,28"
##  [3] "2013-01-03,27.879999,28.469999,27.59,27.77,63140600,27.77"
##  [4] "2013-01-04,28.01,28.93,27.83,28.76,72715400,28.76"
##  [5] "2013-01-07,28.690001,29.790001,28.65,29.42,83781800,29.42"
##  [6] "2013-01-08,29.51,29.6,28.860001,29.059999,45871300,29.059999"
##  [7] "2013-01-09,29.67,30.6,29.49,30.59,104787700,30.59"
##  [8] "2013-01-10,30.6,31.450001,30.280001,31.299999,95316400,31.299999"
##  [9] "2013-01-11,31.280001,31.959999,31.1,31.719999,89598000,31.719999"
## [10] "2013-01-14,32.080002,32.209999,30.620001,30.950001,98892800,30.950001"
```

To import this file, we use `read_csv()` function.

```
read_csv("../data/FB.csv")
```

```
## # A tibble: 5 x 7
##   Index      FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
##   <date>       <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
## 1 2013-01-02    27.4    28.2   27.4     28    69846400        28
## 2 2013-01-03    27.9    28.5   27.6     27.8  63140600        27.8
## 3 2013-01-04    28.0    28.9   27.8     28.8  72715400        28.8
## 4 2013-01-07    28.7    29.8   28.6     29.4  83781800        29.4
## 5 2013-01-08    29.5    29.6   28.9     29.1  45871300        29.1
```

Suppose we have the same data again, however the values are now separated by forwards slash.

```
##  [1] "Index/FB.Open/FB.High/FB.Low/FB.Close/FB.Volume/FB.Adjusted"
##  [2] "2013-01-02/27.440001/28.18/27.42/28/69846400/28"
##  [3] "2013-01-03/27.879999/28.469999/27.59/27.77/63140600/27.77"
##  [4] "2013-01-04/28.01/28.93/27.83/28.76/72715400/28.76"
##  [5] "2013-01-07/28.690001/29.790001/28.65/29.42/83781800/29.42"
##  [6] "2013-01-08/29.51/29.6/28.860001/29.059999/45871300/29.059999"
##  [7] "2013-01-09/29.67/30.6/29.49/30.59/104787700/30.59"
##  [8] "2013-01-10/30.6/31.450001/30.280001/31.299999/95316400/31.299999"
##  [9] "2013-01-11/31.280001/31.959999/31.1/31.719999/89598000/31.719999"
## [10] "2013-01-14/32.080002/32.209999/30.620001/30.950001/98892800/30.950001"
```

In this specific case we use the `read_delim()` function. This is the general function that allows you to import any file in table format. To specify how values are separated, we use the `delim` argument.

```
read_delim("../data/FB_slsh.txt", delim = "/")
```

```
## # A tibble: 10 x 7
##    Index      FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
##    <date>       <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
##  1 2013-01-02    27.4    28.2   27.4     28    69846400        28
##  2 2013-01-03    27.9    28.5   27.6     27.8  63140600        27.8
##  3 2013-01-04    28.0    28.9   27.8     28.8  72715400        28.8
##  4 2013-01-07    28.7    29.8   28.6     29.4  83781800        29.4
##  5 2013-01-08    29.5    29.6   28.9     29.1  45871300        29.1
##  6 2013-01-09    29.7    30.6   29.5     30.6 104787700        30.6
##  7 2013-01-10    30.6    31.5   30.3     31.3  95316400        31.3
##  8 2013-01-11    31.3    32.0   31.1     31.7  89598000        31.7
##  9 2013-01-14    32.1    32.2   30.6     31.0  98892800        31.0
## 10 2013-01-15    30.6    31.7   29.9     30.1 173242600        30.1
```

`read_delim()` function assumes that first row contains column names.

There are several other important arguments in `readr` functions. The `col_names` argument is used for specifying column names. It is `TRUE` by default, which means that values from the first row are recognized as column names. If you set this argument to `FALSE`, random column names would be generated (X1, X2...).

```
read_delim(
  "../data/FB_slsh.txt",
  delim = "/",
  col_names = FALSE
)
```

```
## # A tibble: 6 x 7
##   X1         X2        X3        X4        X5        X6        X7
##   <chr>      <chr>     <chr>     <chr>     <chr>     <chr>     <chr>
## 1 Index      FB.Open   FB.High   FB.Low    FB.Close  FB.Volume FB.Adjusted
## 2 2013-01-02 27.440001 28.18     27.42     28        69846400  28
## 3 2013-01-03 27.879999 28.469999 27.59     27.77     63140600  27.77
## 4 2013-01-04 28.01     28.93     27.83     28.76     72715400  28.76
## 5 2013-01-07 28.690001 29.790001 28.65     29.42     83781800  29.42
## 6 2013-01-08 29.51     29.6      28.860001 29.059999 45871300  29.059999
```

You can also manually define names, inserting you own names inside a character vector.

```
read_delim(
  "../data/FB_slsh.txt",
  delim = "/",
  col_names = c("Index", "FB_open", "FB_high",
                "FB_low", "FB_close", "FB_volume", "FB_adjusted")
)
```

```
## # A tibble: 5 x 7
##   Index      FB_open   FB_high   FB_low FB_close FB_volume FB_adjusted
##   <chr>      <chr>     <chr>     <chr>  <chr>    <chr>     <chr>
## 1 Index      FB.Open   FB.High   FB.Low FB.Close FB.Volume FB.Adjusted
## 2 2013-01-02 27.440001 28.18     27.42  28       69846400  28
## 3 2013-01-03 27.879999 28.469999 27.59  27.77    63140600  27.77
## 4 2013-01-04 28.01     28.93     27.83  28.76    72715400  28.76
## 5 2013-01-07 28.690001 29.790001 28.65  29.42    83781800  29.42
```

With `col_types` you can specify the types of columns in your data frame. By default it is set to `NULL`, which means that R automatically guesses the types based on first several rows in the data frame. You can manually set column types inside a character vector by writing a character (`c`, `n`, `d`, `i`, `l`, `D`) that represents one column type (`character`, `number`, `double`, `integer`, `logical`, `date`). You can also use an underscore _ to skip a column.

```
read_csv(
  "../data/FB.csv",
  col_types = c("cdddddd")
)
```

```
## # A tibble: 10 x 7
##    Index      FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
##    <chr>        <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
##  1 2013-01-02    27.4    28.2   27.4     28    69846400        28
##  2 2013-01-03    27.9    28.5   27.6     27.8  63140600        27.8
##  3 2013-01-04    28.0    28.9   27.8     28.8  72715400        28.8
##  4 2013-01-07    28.7    29.8   28.6     29.4  83781800        29.4
##  5 2013-01-08    29.5    29.6   28.9     29.1  45871300        29.1
##  6 2013-01-09    29.7    30.6   29.5     30.6 104787700        30.6
##  7 2013-01-10    30.6    31.5   30.3     31.3  95316400        31.3
##  8 2013-01-11    31.3    32.0   31.1     31.7  89598000        31.7
##  9 2013-01-14    32.1    32.2   30.6     31.0  98892800        31.0
## 10 2013-01-15    30.6    31.7   29.9     30.1 173242600        30.1
```

Sometimes you may want to skip first n rows or to select just first n records because you do not need the rest of the data. The `readr` package allows you to do that by using `skip` and `n_max` arguments.

```
read_csv(
  "../data/FB.csv",
  skip = 5,
  n_max = 4
)
```

```
## # A tibble: 4 x 7
##   `2013-01-08` `29.51` `29.6` `28.860001` `29.059999` `45871300` `29.059999_1`
##   <date>         <dbl>  <dbl>       <dbl>       <dbl>      <dbl>         <dbl>
## 1 2013-01-09      29.7   30.6        29.5        30.6  104787700          30.6
## 2 2013-01-10      30.6   31.5        30.3        31.3   95316400          31.3
## 3 2013-01-11      31.3   32.0        31.1        31.7   89598000          31.7
## 4 2013-01-14      32.1   32.2        30.6        31.0   98892800          31.0
```

You see that first 5 rows are skipped and only next 4 rows are imported. However, since `col_names` argument is set to `TRUE` by default, the fifth row is recognized as our column names. So, we need to set this argument to `FALSE` or specify it manually.

```
read_csv(
  "../data/FB.csv",
  skip = 5,
  n_max = 4,
  col_names = c(
    "Index", "FB_open", "FB_high", "FB_low",
    "FB_close", "FB_volume", "FB_adjusted"
  )
)
```

```
## # A tibble: 4 x 7
##   Index      FB_open FB_high FB_low FB_close FB_volume FB_adjusted
##   <date>       <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
## 1 2013-01-08    29.5    29.6   28.9     29.1  45871300        29.1
## 2 2013-01-09    29.7    30.6   29.5     30.6 104787700        30.6
## 3 2013-01-10    30.6    31.5   30.3     31.3  95316400        31.3
## 4 2013-01-11    31.3    32.0   31.1     31.7  89598000        31.7
```

Importing Excel data

Excel files can be imported using functions from various packages. However, the most convenient way to do it is by using functions from the `readxl` package. An excel file contains different sheets with tabular data. Let's see one example. Before importing an excel file into R, you will probably firstly explore it.

A1 | Index

	A	B	C	D	E	F	G
1	Index	FB.Open	FB.High	FB.Low	FB.Close	FB.Volume	FB.Adjusted
2	02/01/13	27.440001	28.18	27.42	28	69846400	28
3	03/01/13	27.879999	28.469999	27.59	27.77	63140600	27.77
4	04/01/13	28.01	28.93	27.83	28.76	72715400	28.76
5	07/01/13	28.690001	29.790001	28.65	29.42	83781800	29.42
6	08/01/13	29.51	29.6	28.860001	29.059999	45871300	29.059999
7	09/01/13	29.67	30.6	29.49	30.59	104787700	30.59
8	10/01/13	30.6	31.450001	30.280001	31.299999	95316400	31.299999
9	11/01/13	31.280001	31.959999	31.1	31.719999	89598000	31.719999
10	14/01/13	32.080002	32.209999	30.620001	30.950001	98892800	30.950001
11	15/01/13	30.639999	31.709999	29.879999	30.1	173242600	30.1
12	16/01/13	30.209999	30.35	29.530001	29.85	75332700	29.85
13	17/01/13	30.08	30.42	30.030001	30.139999	40256700	30.139999
14	18/01/13	30.309999	30.440001	29.27	29.66	49631500	29.66
15	22/01/13	29.75	30.889999	29.74	30.73	55243300	30.73
16	23/01/13	31.1	31.5	30.799999	30.82	48899800	30.82
17	24/01/13	31.27	31.49	30.809999	31.08	43845100	31.08
18	25/01/13	31.41	31.93	31.129999	31.540001	54363600	31.540001
19	28/01/13	31.879999	32.509998	31.809999	32.470001	59682500	32.470001
20	29/01/13	32	32.07	30.709999	30.790001	72976500	30.790001
21	30/01/13	30.98	31.49	30.879999	31.24	87682100	31.24
22	31/01/13	29.15	31.469999	28.74	30.98	190744900	30.98
23	01/02/13	31.01	31.02	29.629999	29.73	85856700	29.73
24	04/02/13	29.059999	29.200001	28.01	28.110001	92362200	28.110001
25	05/02/13	28.26	28.959999	28.040001	28.639999	47948200	28.639999
26	06/02/13	28.74	29.290001	28.66	29.049999	38375900	29.049999
27	07/02/13	29.110001	29.15	28.27	28.65	34540100	28.65
28	08/02/13	28.889999	29.17	28.51	28.549999	37708800	28.549999
29	11/02/13	28.610001	28.68	28.040001	28.26	37361800	28.26
30	12/02/13	27.67	28.16	27.1	27.370001	93498700	27.370001
31	13/02/13	27.360001	28.32	27.309999	27.91	50164000	27.91
32	14/02/13	28.02	28.629999	28.01	28.5	35615800	28.5

FB | AAPL

C1

	A	B	C	D	E	F	G	H
1	x							
2								
3		Index	AAPL.Open	AAPL.High	AAPL.Low	AAPL.Close	AAPL.Volume	AAPL.Adjusted
4		02/01/13	19.779285	19.821428	19.343929	19.608213	560518000	17.094694
5		03/01/13	19.567142	19.631071	19.321428	19.360714	352965200	16.87892
6		04/01/13	19.1775	19.236786	18.779642	18.821428	594333600	16.408764
7		07/01/13	18.642857	18.903572	18.4	18.710714	484156400	16.312239
8		08/01/13	18.900356	18.996071	18.616072	18.761072	458707200	16.35615
9		09/01/13	18.660715	18.750357	18.428215	18.467857	407604400	16.100521
10		10/01/13	18.876785	18.882856	18.411428	18.696785	601146000	16.3001
11		11/01/13	18.607143	18.761429	18.536428	18.582144	350506800	16.200155
12		14/01/13	17.952858	18.125	17.803928	17.919643	734207600	15.622575
13		15/01/13	17.796429	17.821072	17.263571	17.354286	876772400	15.129693
14		16/01/13	17.665714	18.194286	17.589285	18.074642	690804800	15.757706
15		17/01/13	18.225357	18.241072	17.929644	17.952858	453678400	15.651527
16		18/01/13	17.804285	17.936428	17.728571	17.857143	472922800	15.568091
17		22/01/13	18.02	18.138571	17.736786	18.0275	461546400	15.716609
18		23/01/13	18.171785	18.3925	18.0275	18.3575	861509600	16.004307
19		24/01/13	16.428572	16.633215	16.080357	16.089285	1460852400	14.02685
20		25/01/13	16.131786	16.293928	15.535714	15.71	1208026400	13.696181
21		28/01/13	15.636786	16.186071	15.566428	16.065357	785517600	14.005986
22		29/01/13	16.375	16.435715	16.147142	16.366785	571158000	14.268773
23		30/01/13	16.321428	16.521429	16.232143	16.315357	417155200	14.223937
24		31/01/13	16.320715	16.402857	16.249287	16.2675	319334400	14.182214
25		01/02/13	16.396786	16.41	16.012501	16.200714	539484400	14.12399
26		04/02/13	16.211071	16.283571	15.785714	15.797143	477117200	13.772153
27		05/02/13	15.858929	16.419285	15.793571	16.351429	573347600	14.255387
28		06/02/13	16.3025	16.660715	16.163572	16.333929	593706400	14.240132
29		07/02/13	16.544643	16.785715	16.218571	16.722143	704580800	14.663541
30		08/02/13	16.928572	17.100357	16.723215	16.963572	633158400	14.875249
31		11/02/13	17.017857	17.319286	16.901785	17.140358	517490400	15.03027
32		12/02/13	17.125357	17.227858	16.705	16.710714	609053200	14.65352

FB | AAPL

We see that there are two sheets that include information about stock prices of two different companies, Facebook and Apple. We can also see that there are some irrelevant rows and columns in the second sheet that we would rather skip.

With the `excel_sheets()` function you can check how many sheets are in your excel file, and what are their names. There is only one argument which is the path to directory where your excel file is saved.

```
library(readxl)
excel_sheets("../data/stocks.xlsx")
```

```
## [1] "FB"   "AAPL"
```

The next step is importing the data. To import an excel file we use the `read_excel()` function. Assume that we want to import information about the Facebook stocks. Again, we have to determine the path to the file. You can call the function without specifying other arguments. However, if you don't specify the `sheet` argument, first sheet would be inserted by default. Inside `sheet` argument you can both use an index or the sheet name.

```
read_excel("../data/stocks.xlsx", sheet = "FB")
```

```
## # A tibble: 10 x 7
##    Index                FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
##    <dttm>                 <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
##  1 2013-01-02 00:00:00    27.4    28.2   27.4     28    69846400        28
##  2 2013-01-03 00:00:00    27.9    28.5   27.6     27.8  63140600        27.8
##  3 2013-01-04 00:00:00    28.0    28.9   27.8     28.8  72715400        28.8
##  4 2013-01-07 00:00:00    28.7    29.8   28.6     29.4  83781800        29.4
##  5 2013-01-08 00:00:00    29.5    29.6   28.9     29.1  45871300        29.1
##  6 2013-01-09 00:00:00    29.7    30.6   29.5     30.6 104787700        30.6
##  7 2013-01-10 00:00:00    30.6    31.5   30.3     31.3  95316400        31.3
##  8 2013-01-11 00:00:00    31.3    32.0   31.1     31.7  89598000        31.7
##  9 2013-01-14 00:00:00    32.1    32.2   30.6     31.0  98892800        31.0
## 10 2013-01-15 00:00:00    30.6    31.7   29.9     30.1 173242600        30.1
```

Apart from the `path` and `sheet` arguments, there are other arguments you can specify. Similar to `readr` functions, there are `col_names` and `col_types` arguments used for defining column names and types.

The `col_names` argument is `TRUE` by default, which means that column names are in the first row of the excel file. If it is set to `FALSE`, R assigns its own names. Finally, you can manually specify column names with a character vector.

To define column types you use the `col_types` argument. Default is `NULL` in which case R guesses column types. However, you can set column types manually. Excel can contain different data types, such as text, numerics and dates which you define by writing `text`, `numeric` and `date` inside the character vector. You can also skip a column by using the `skip` option inside the abovementioned character vector.

There is also a separate `skip` argument which is used to specify the number of rows in your excel sheet R will skip before importing the data. The `n_max` argument is used again for reading the n number of rows. In order to limit the read to a specific rectangle of cells you use the `range` argument (e.g. `range = "D12:F15"`).

```
read_excel("../data/stocks.xlsx", sheet = "AAPL",
           col_types = c(
             "date", "numeric", "numeric", "numeric",
             "numeric", "numeric", "numeric"
           ),
           range = "B3:H1262")
```

```
## # A tibble: 10 x 7
##    Index               AAPL.Open AAPL.High AAPL.Low AAPL.Close AAPL.Volume
##    <dttm>                  <dbl>     <dbl>    <dbl>      <dbl>       <dbl>
##  1 2013-01-02 00:00:00      19.8      19.8     19.3       19.6   560518000
##  2 2013-01-03 00:00:00      19.6      19.6     19.3       19.4   352965200
##  3 2013-01-04 00:00:00      19.2      19.2     18.8       18.8   594333600
##  4 2013-01-07 00:00:00      18.6      18.9     18.4       18.7   484156400
##  5 2013-01-08 00:00:00      18.9      19.0     18.6       18.8   458707200
##  6 2013-01-09 00:00:00      18.7      18.8     18.4       18.5   407604400
##  7 2013-01-10 00:00:00      18.9      18.9     18.4       18.7   601146000
##  8 2013-01-11 00:00:00      18.6      18.8     18.5       18.6   350506800
##  9 2013-01-14 00:00:00      18.0      18.1     17.8       17.9   734207600
## 10 2013-01-15 00:00:00      17.8      17.8     17.3       17.4   876772400
## # ... with 1 more variable: AAPL.Adjusted <dbl>
```

Importing financial and economical data

In this chapter we will learn how to import financial and economic data using the functions from the `quantmod` and the `Quandl` packages. With the `getSymbols()` function from the `quantmod` package you can import financial data from multiple data sources. By default, `getSymbols()` stores data in an `xts` object[4], but it can use other classes as well.

[4] We would talk about `xts` objects at the end of this course

The first argument of the `getSymbols()` function is `Symbols`, where you specify the instrument you want to import which can be a stock ticker symbol, an exchange rate, an economic data series, or some other instrument. The second important argument is the `src` argument, which tells the function which data source to import data from. The default source is `yahoo`. However `getSymbols` function supports many other sources as well, such as:

- Yahoo Finance
- Google Finance
- Federal Reserve Economic Data ("FRED")
- Oanda (foreign exchanges)
- CSV files
- MySQL
- SQLite
- RDS files

`getSymbols` can import data in two ways. With the `auto.assign` argument we control whether the data is returned or loaded. The default argument is `TRUE` which means that data is automatically loaded into your workspace. However, if we want to assign it yourself to an object, we must specify this argument to `FALSE`. Also, if you want your data to be imported as `data.frame` object, you can set `return.class` argument to `data.frame`. Now, let's see where all stocks information we previously imported come from.

```r
library(quantmod)
getSymbols(Symbols = "FB")
```

```
## [1] "FB"
```

```r
head(FB)
```

```
##            FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
## 2012-05-18   42.05   45.00  38.00    38.23 573576400       38.23
## 2012-05-21   36.53   36.66  33.00    34.03 168192700       34.03
## 2012-05-22   32.61   33.59  30.94    31.00 101786600       31.00
## 2012-05-23   31.37   32.50  31.36    32.00  73600000       32.00
## 2012-05-24   32.95   33.21  31.77    33.03  50237200       33.03
## 2012-05-25   32.90   32.95  31.11    31.91  37149800       31.91
```

```r
fb_stock <- getSymbols("FB", auto.assign = FALSE, return.class = "data.frame")
head(fb_stock)
```

```
##            FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
## 2012-05-18   42.05   45.00  38.00    38.23 573576400       38.23
## 2012-05-21   36.53   36.66  33.00    34.03 168192700       34.03
## 2012-05-22   32.61   33.59  30.94    31.00 101786600       31.00
## 2012-05-23   31.37   32.50  31.36    32.00  73600000       32.00
## 2012-05-24   32.95   33.21  31.77    33.03  50237200       33.03
## 2012-05-25   32.90   32.95  31.11    31.91  37149800       31.91
```

```r
gdp <- getSymbols(
  Symbols = "GDP",
  src = "FRED",
  auto.assign = FALSE,
  return.class = "data.frame"
)
head(gdp)
```

```
##                 GDP
## 1947-01-01 243.164
## 1947-04-01 245.968
## 1947-07-01 249.585
## 1947-10-01 259.745
## 1948-01-01 265.742
## 1948-04-01 272.567
```

The `Quandl()` function from the `Quandl` package is another function used for importing financial and economic data. This function is the part of the `Quandl` package. The main difference between `getSymbols()` and `Quandl()` functions is that the second one always returns data it imports, so you do not need to specify additional argument `auto.assign` as in the `getSymbols()` function. Also, in the `Quandl()` function you specify the instrument and the data source inside only one argument called `code`. Another important difference is that `Quandl()` returns a `data.frame` by default, not `xts`, which can be modified as well.

```
library(Quandl)
gdp_quandl <- Quandl("FRED/GDP")
head(gdp_quandl)
```

```
##         Date    Value
## 1 2021-07-01 23202.34
## 2 2021-04-01 22740.96
## 3 2021-01-01 22038.23
## 4 2020-10-01 21477.60
## 5 2020-07-01 21138.57
## 6 2020-04-01 19477.44
```

Subsection 2

Working with data

Data Manipulation with dplyr

We have already mentioned the very powerful set of tools for exploring data in R called `tidyverse`. The `dplyr` package is one of the core packages of the `tidyverse`. It includes various functions designed to facilitate manipulation of data in an intuitive, user-friendly way.

There are a lot of important `dplyr` functions that are used on everyday basis. Most commonly used functions are `select()`, `mutate()`, `filter()` and `arrange()`. With the `select()` function we extract only particular **columns** needed for further analysis. To add column(s) in a dataframe we use the `mutate()` command. You can use the `filter()` function to extract only particular **observations** from a dataset, based on a single or multiple conditions. The `arrange()` function sorts data based on one or more variables in an ascending or descending order in which case we need to wrap the name of the column with the `desc()` function.

In this chapter we would use a dataset that consists of information about customers that applied for a home loan[5]. Dataset includes categorical variables such as gender, marital status and education, as well as continuous variables such as an information about applicant's income and loan amount.

```
df <- read_csv("../data/loan_status.csv")
head(df, 10)
```

```
## # A tibble: 10 x 13
##    Loan_ID Gender Married Dependents Education Self_Employed ApplicantIncome
##    <chr>   <chr>  <chr>   <chr>      <chr>     <chr>                   <dbl>
##  1 LP0010~ Male   No      0          Graduate  No                       5849
##  2 LP0010~ Male   Yes     1          Graduate  No                       4583
##  3 LP0010~ Male   Yes     0          Graduate  Yes                      3000
##  4 LP0010~ Male   Yes     0          Not Grad~ No                       2583
##  5 LP0010~ Male   No      0          Graduate  No                       6000
##  6 LP0010~ Male   Yes     2          Graduate  Yes                      5417
##  7 LP0010~ Male   Yes     0          Not Grad~ No                       2333
##  8 LP0010~ Male   Yes     3+         Graduate  No                       3036
##  9 LP0010~ Male   Yes     2          Graduate  No                       4006
## 10 LP0010~ Male   Yes     1          Graduate  No                      12841
## # ... with 6 more variables: CoapplicantIncome <dbl>, LoanAmount <dbl>,
## #   Loan_Amount_Term <dbl>, Credit_History <dbl>, Property_Area <chr>,
## #   Loan_Status <chr>
```

[5] The dataset is downloaded from *https://www.kaggle.com/sethirishabh/finance-company-loan-data*

We see that there are a lot of variables. Let's select just columns I mentioned.

```
df_v1 <- select(
  df, Loan_ID, Gender, Married, Education, ApplicantIncome, LoanAmount
)
head(df_v1, 10)
```

```
## # A tibble: 10 x 6
##    Loan_ID  Gender Married Education    ApplicantIncome LoanAmount
##    <chr>    <chr>  <chr>   <chr>                  <dbl>      <dbl>
##  1 LP001002 Male   No      Graduate                5849         NA
##  2 LP001003 Male   Yes     Graduate                4583        128
##  3 LP001005 Male   Yes     Graduate                3000         66
##  4 LP001006 Male   Yes     Not Graduate            2583        120
##  5 LP001008 Male   No      Graduate                6000        141
##  6 LP001011 Male   Yes     Graduate                5417        267
##  7 LP001013 Male   Yes     Not Graduate            2333         95
##  8 LP001014 Male   Yes     Graduate                3036        158
##  9 LP001018 Male   Yes     Graduate                4006        168
## 10 LP001020 Male   Yes     Graduate               12841        349
```

With the `select()` function we can also omit columns by using the minus sign before the name of the column we do not need.

```
df_omit <- select(df_v1, -Gender)
head(df_omit, 10)
```

```
## # A tibble: 10 x 5
##    Loan_ID  Married Education    ApplicantIncome LoanAmount
##    <chr>    <chr>   <chr>                  <dbl>      <dbl>
##  1 LP001002 No      Graduate                5849         NA
##  2 LP001003 Yes     Graduate                4583        128
##  3 LP001005 Yes     Graduate                3000         66
##  4 LP001006 Yes     Not Graduate            2583        120
##  5 LP001008 No      Graduate                6000        141
##  6 LP001011 Yes     Graduate                5417        267
##  7 LP001013 Yes     Not Graduate            2333         95
##  8 LP001014 Yes     Graduate                3036        158
##  9 LP001018 Yes     Graduate                4006        168
## 10 LP001020 Yes     Graduate               12841        349
```

In addition, you can use so called **selection helpers** inside the `select()` function, such as:

- `everything()` - matches all variables,
- `last_col()` - selects the last variable,
- `starts_with()` - starts with a defined prefix (patterns in names),
- `ends_with()` - ends with a defined suffix,
- `contains()` - contains a string we define,

```
df_slct_help <- select(df_v1, starts_with("Loan"))
head(df_slct_help, 6)
```

```
## # A tibble: 6 x 2
##   Loan_ID  LoanAmount
##   <chr>         <dbl>
## 1 LP001002         NA
## 2 LP001003        128
## 3 LP001005         66
## 4 LP001006        120
## 5 LP001008        141
## 6 LP001011        267
```

Lastly, instead of writing column names inside the `select()` function, you can also use column position to indicate columns you wish to select.

Assume now we want to calculate the ratio between the loan amount and the applicant income and add it as a new variable.

```
df_v2 <- mutate(df_v1, Ratio = LoanAmount / ApplicantIncome)
head(df_v2, 10)
```

```
## # A tibble: 10 x 7
##    Loan_ID  Gender Married Education    ApplicantIncome LoanAmount   Ratio
##    <chr>    <chr>  <chr>   <chr>                  <dbl>      <dbl>   <dbl>
##  1 LP001002 Male   No      Graduate                5849         NA NA
##  2 LP001003 Male   Yes     Graduate                4583        128  0.0279
##  3 LP001005 Male   Yes     Graduate                3000         66  0.022
##  4 LP001006 Male   Yes     Not Graduate            2583        120  0.0465
##  5 LP001008 Male   No      Graduate                6000        141  0.0235
##  6 LP001011 Male   Yes     Graduate                5417        267  0.0493
##  7 LP001013 Male   Yes     Not Graduate            2333         95  0.0407
##  8 LP001014 Male   Yes     Graduate                3036        158  0.0520
##  9 LP001018 Male   Yes     Graduate                4006        168  0.0419
## 10 LP001020 Male   Yes     Graduate               12841        349  0.0272
```

We might be interested only in graduate applicants. In that case, we need to use the `filter()` function. Recall that we use double equal sign == as an equality operator.

```
df_v3 <- filter(df_v2, Education == "Graduate")
head(df_v3, 10)
```

```
## # A tibble: 10 x 7
##    Loan_ID  Gender Married Education ApplicantIncome LoanAmount   Ratio
##    <chr>    <chr>  <chr>   <chr>               <dbl>      <dbl>   <dbl>
##  1 LP001002 Male   No      Graduate             5849         NA NA
##  2 LP001003 Male   Yes     Graduate             4583        128  0.0279
##  3 LP001005 Male   Yes     Graduate             3000         66  0.022
##  4 LP001008 Male   No      Graduate             6000        141  0.0235
##  5 LP001011 Male   Yes     Graduate             5417        267  0.0493
##  6 LP001014 Male   Yes     Graduate             3036        158  0.0520
##  7 LP001018 Male   Yes     Graduate             4006        168  0.0419
##  8 LP001020 Male   Yes     Graduate            12841        349  0.0272
##  9 LP001024 Male   Yes     Graduate             3200         70  0.0219
## 10 LP001027 Male   Yes     Graduate             2500        109  0.0436
```

If we are interested to see which customers have the lowest or highest income we would sort the table by the `ApplicantIncome` column. Remember that you need to use the `desc()` function inside `arrange()` in case you wamt to sort the table in descending order.

```
df_v4a <- arrange(df_v3, ApplicantIncome)
head(df_v4a, 6)
```

```
## # A tibble: 6 x 7
##   Loan_ID  Gender Married Education ApplicantIncome LoanAmount Ratio
##   <chr>    <chr>  <chr>   <chr>               <dbl>      <dbl> <dbl>
## 1 LP001722 Male   Yes     Graduate              150        135 0.9
## 2 LP002949 Female No      Graduate              416        350 0.841
## 3 LP002603 Female No      Graduate              645        113 0.175
## 4 LP001644 <NA>   Yes     Graduate              674        168 0.249
## 5 LP001259 Male   Yes     Graduate             1000        110 0.11
## 6 LP002345 Male   Yes     Graduate             1025        112 0.109
```

```
df_v4b <- arrange(df_v3, desc(ApplicantIncome))
head(df_v4b, 6)
```

```
## # A tibble: 6 x 7
##   Loan_ID  Gender Married Education ApplicantIncome LoanAmount   Ratio
##   <chr>    <chr>  <chr>   <chr>               <dbl>      <dbl>   <dbl>
## 1 LP002317 Male   Yes     Graduate            81000        360 0.00444
## 2 LP002101 Male   Yes     Graduate            63337        490 0.00774
## 3 LP001585 <NA>   Yes     Graduate            51763        700 0.0135
## 4 LP001536 Male   Yes     Graduate            39999        600 0.0150
## 5 LP001640 Male   Yes     Graduate            39147        120 0.00307
## 6 LP002422 Male   No      Graduate            37719        152 0.00403
```

Piping

Before we continue with other important `dplyr` functions, we introduce the incredibly clever and effective way to combine multiple operations. This concept allows you to simplify your code and make your operations more intuitive. Pipes are written using the combination of signs - %>%. This useful tool is part of the `magrittr` package, which is exported to `dplyr` and other `tidyverse` packages. It may be confusing at the beginning, but extremely useful when you get used to it.

Let's now perform all operations from previous examples in one step using pipes.

```
df %>%
  select(Loan_ID, Gender, Married, Education, ApplicantIncome, LoanAmount) %>%
  mutate(Ratio = LoanAmount / ApplicantIncome) %>%
  filter(Education == "Graduate") %>%
  arrange(ApplicantIncome) %>%
  head(n = 10)
```

```
## # A tibble: 10 x 7
##    Loan_ID  Gender Married Education ApplicantIncome LoanAmount  Ratio
##    <chr>    <chr>  <chr>   <chr>               <dbl>      <dbl>  <dbl>
##  1 LP001722 Male   Yes     Graduate              150        135 0.9
##  2 LP002949 Female No      Graduate              416        350 0.841
##  3 LP002603 Female No      Graduate              645        113 0.175
##  4 LP001644 <NA>   Yes     Graduate              674        168 0.249
##  5 LP001259 Male   Yes     Graduate             1000        110 0.11
##  6 LP002345 Male   Yes     Graduate             1025        112 0.109
##  7 LP002717 Male   Yes     Graduate             1025        216 0.211
##  8 LP001030 Male   Yes     Graduate             1299         17 0.0131
##  9 LP001267 Female Yes     Graduate             1378        167 0.121
## 10 LP002287 Female No      Graduate             1500        103 0.0687
```

As you see, R sequentially does one by one operation, step by step. First argument is not needed in `dplyr` functions, as output is automatically passed from the previous line into the next line as input.

More dplyr functions

The `dplyr` package includes many other important functions. Sometimes, you might need to rename columns of the dataset. Luckily, the `rename()` function enables you do that.

Also, you may be interested in finding distinct categories of some specific column with the `distinct()` function.

Previously, we saw that we can filter observations based on defined conditions with the `filter()` function. Furthermore, you can filter observations by their integer locations with the `slice()` function.

While `mutate()` adds new variables and preserves existing ones, `transmute()` at the same time adds new variables and removes existing ones. If new variable has the same name as one of the existing variables, new variable overwrites the existing one.

With the `pull()` function you can extract desired column. You can do that either by writing the name of the column, or its integer position.

It is time for using these functions in examples. Of course, we would again use pipes!

```
df %>%
  rename(Education_status = Education) %>%
  distinct(Education_status)
```

```
## # A tibble: 2 x 1
##   Education_status
##   <chr>
## 1 Graduate
## 2 Not Graduate
```

```
(x <- df %>%
  transmute(
    Loan_ID = Loan_ID,
    Education_status = Education,
    Ratio = LoanAmount / ApplicantIncome
  ) %>% slice(1:6)
 )
```

```
## # A tibble: 6 x 3
##   Loan_ID  Education_status   Ratio
##   <chr>    <chr>              <dbl>
## 1 LP001002 Graduate         NA
## 2 LP001003 Graduate          0.0279
## 3 LP001005 Graduate          0.022
## 4 LP001006 Not Graduate      0.0465
## 5 LP001008 Graduate          0.0235
## 6 LP001011 Graduate          0.0493
```

```
x %>%
  pull(Loan_ID)
```

```
## [1] "LP001002" "LP001003" "LP001005" "LP001006" "LP001008" "LP001011"
```

Subsection 3

Cleaning and transforming data

Cleaning data

Earlier, we learned to convert variable types when reading data using the `col_types` argument inside the `readr` package functions so that the variable type stored in R matches the values. Alternatively, you may let R guess column types by leaving the `col_types` argument empty. In that case you might encounter several problems and find that not all columns are properly guessed. Also, sometimes data is not properly entered or formatted, so you have to perform certain corrections in order to make your data ready for further analysis.

Take a look at the following example.

```
df_unclean <- read_csv("../data/loan_status_unclean.csv")
df_unclean %>% head(n = 10)
```

```
## # A tibble: 10 x 7
##    Month      Loan_ID  Gender Married ApplicantIncome Credit_History Loan_Status
##    <chr>      <chr>    <chr>  <chr>   <chr>                    <dbl> <chr>
##  1 2020-1-01 LP001002 Male   No      $5,849                       1 Y
##  2 2020-1-02 LP001003 Male   Yes     $4,583                       1 N
##  3 2020-1-03 LP001005 Male   Yes     $3,000                       1 Y
##  4 2020-1-04 LP001006 Male   Yes     $2,583                       1 Y
##  5 2020-1-05 LP001008 Male   No      $6,000                       1 Y
##  6 2020-1-06 LP001011 Male   Yes     $5,417                       1 Y
##  7 2020-1-07 LP001013 Male   Yes     $2,333                       1 Y
##  8 2020-1-08 LP001014 Male   Yes     $3,036                       0 N
##  9 2020-1-09 LP001018 Male   Yes     $4,006                       1 Y
## 10 2020-1-10 LP001020 Male   Yes     $12,841                      1 N
```

We saw that the Month column was recognized as string. It is because the month (January in this example) is written with a single digit without the leading zero. This can be changed using the `as.Date()` function from the base package.

```
df_unclean %>% mutate(Month = as.Date(Month)) %>% head(n = 10)
```

```
## # A tibble: 10 x 7
##    Month      Loan_ID  Gender Married ApplicantIncome Credit_History Loan_Status
##    <date>     <chr>    <chr>  <chr>   <chr>                    <dbl> <chr>
##  1 2020-01-01 LP001002 Male   No      $5,849                       1 Y
##  2 2020-01-02 LP001003 Male   Yes     $4,583                       1 N
##  3 2020-01-03 LP001005 Male   Yes     $3,000                       1 Y
##  4 2020-01-04 LP001006 Male   Yes     $2,583                       1 Y
##  5 2020-01-05 LP001008 Male   No      $6,000                       1 Y
##  6 2020-01-06 LP001011 Male   Yes     $5,417                       1 Y
##  7 2020-01-07 LP001013 Male   Yes     $2,333                       1 Y
##  8 2020-01-08 LP001014 Male   Yes     $3,036                       0 N
##  9 2020-01-09 LP001018 Male   Yes     $4,006                       1 Y
## 10 2020-01-10 LP001020 Male   Yes     $12,841                      1 N
```

However, much easier way to work with dates and times is to use functions from the `lubridate` package. The `lubridate` package contains `as_date()` function which works in a similar way as the `as.Date()` function (negligible differences). With `ymd()`, `ydm()`, `mdy()`, `myd()`, `dmy()`, `dym()` functions you can import heterogeneous date formats. Apart from `as_date()` and `ymd()` family functions, the `lubridate` package enables you to extract a weekday or quarter from the date, round the date-time object down or up to the nearest boundary of the specified time unit, add days/months/years, or present date object in different formats.

```r
library(lubridate)
df_unclean %>%
  transmute(
    Month = as_date(Month), #as_date
    Month_ymd = ymd(Month), #ymd
    Month_start = floor_date(Month, "month"), #floor_date
    Add_day = Month + days(3), #adding days
    Wday = wday(Month), #weekday
    Wday_label = wday(Month, label = T), #weekday label
    Quarter = quarter(Month) #quarter
  ) %>% head(n = 10)
```

```
## # A tibble: 10 x 7
##    Month      Month_ymd  Month_start Add_day     Wday Wday_label Quarter
##    <date>     <date>     <date>      <date>     <dbl> <ord>        <int>
##  1 2020-01-01 2020-01-01 2020-01-01  2020-01-04     4 Wed              1
##  2 2020-01-02 2020-01-02 2020-01-01  2020-01-05     5 Thu              1
##  3 2020-01-03 2020-01-03 2020-01-01  2020-01-06     6 Fri              1
##  4 2020-01-04 2020-01-04 2020-01-01  2020-01-07     7 Sat              1
##  5 2020-01-05 2020-01-05 2020-01-01  2020-01-08     1 Sun              1
##  6 2020-01-06 2020-01-06 2020-01-01  2020-01-09     2 Mon              1
##  7 2020-01-07 2020-01-07 2020-01-01  2020-01-10     3 Tue              1
##  8 2020-01-08 2020-01-08 2020-01-01  2020-01-11     4 Wed              1
##  9 2020-01-09 2020-01-09 2020-01-01  2020-01-12     5 Thu              1
## 10 2020-01-10 2020-01-10 2020-01-01  2020-01-13     6 Fri              1
```

Categorical variables are treated differently in statistical models you develop and useful in any kind of data analysis. They are useful in columns which have a limited number of unique values. In our example, the *Gender*, *Married* and *Loan_Status* are columns of that kind. We have already mentioned the `factor()` function that is used for defining categorical variables. If we want to change the column type, we use the `as.factor()` function.

```
df_unclean %>%
  transmute(
    Gender = as.factor(Gender),
    Married = as.factor(Married),
    Loan_Status = as.factor(Loan_Status)
  ) %>% head(n = 6)
```

```
## # A tibble: 6 x 3
##   Gender Married Loan_Status
##   <fct>  <fct>   <fct>
## 1 Male   No      Y
## 2 Male   Yes     N
## 3 Male   Yes     Y
## 4 Male   Yes     Y
## 5 Male   No      Y
## 6 Male   Yes     Y
```

The ApplicantIncome column is recognized as string due to the dollar sign and comma separator. Firstly, we need to eliminate these characters and then convert the column to numeric. The package that contains functions which handle the most common string manipulations is the `stringr` package. Before overcoming previously mentioned issue, let me list some important `stringr` functions that are most commonly used:

- `str_c` - joins multiple vectors into a single vector,
- `str_count` - counts occurrences of a predefined pattern,
- `str_detect` - checks whether a string contains predefined pattern,
- `str_remove` - removes predefined pattern,
- `str_replace` - replaces string pattern to some other pattern,
- `str_extract` - extracts string base on predefined pattern,
- `str_to_lower`, `str_to_upper`, `str_to_sentence`, `str_to_title` - converts case of a string,
- `str_sub` - extracts substring based on a start and end position.

There are several ways to fix the ApplicantIncome column.

```
library(stringr)
df_unclean %>%
  transmute(
    ApplicantIncome = ApplicantIncome,
    rem_str = str_remove(str_remove(ApplicantIncome, ","), "\\$"),
    repl_str = str_replace(str_replace(ApplicantIncome, ",", ""), "\\$", ""),
    comb_str = str_remove(str_sub(ApplicantIncome, 2, str_count(ApplicantIncome)), ",")
  ) %>% head(n = 10)
```

```
## # A tibble: 10 x 4
##    ApplicantIncome rem_str repl_str comb_str
##    <chr>           <chr>   <chr>    <chr>
##  1 $5,849          5849    5849     5849
##  2 $4,583          4583    4583     4583
##  3 $3,000          3000    3000     3000
##  4 $2,583          2583    2583     2583
##  5 $6,000          6000    6000     6000
##  6 $5,417          5417    5417     5417
##  7 $2,333          2333    2333     2333
##  8 $3,036          3036    3036     3036
##  9 $4,006          4006    4006     4006
## 10 $12,841         12841   12841    12841
```

Keep in mind when forming a regular expression[6] that $ is a special character. That is why two backslashes stand before the dollar sign.

[6] A regular expression (shortened as **regex**) is a sequence of characters that define a search pattern in many programming languages.

First two examples can be simplified by defining a pattern that would remove both , and $ at the same time, and by calling only one function from the `stringr` package. However, this time `str_remove()` and `str_replace()` functions should be replaced by `str_remove_all()` and `str_replace_all()` which eliminate all cases that satisfy predefined patterns.

```
df_unclean %>%
  transmute(
    rem_all_str = str_remove_all(ApplicantIncome, "(,|\\$)"),
    repl_all_str = str_replace_all(ApplicantIncome, "(,|\\$)", "")
  ) %>% head(n = 10)
```

```
## # A tibble: 10 x 2
##    rem_all_str repl_all_str
##    <chr>       <chr>
##  1 5849        5849
##  2 4583        4583
##  3 3000        3000
##  4 2583        2583
##  5 6000        6000
##  6 5417        5417
##  7 2333        2333
##  8 3036        3036
##  9 4006        4006
## 10 12841       12841
```

The last step is converting the `ApplicantIncome` column to numeric.

Regular expressions give us a range of possibilities to play with strings. Instead of writing each individual character inside a pattern, one can use `\\D`, which matches anything other than a decimal digit. However, regex is a concept which is out of scope for this lecture.[7]

[7] To learn and try different regular expressions visit *https://regex101.com/*.

The final table would look like this:

```
df_clean <- df_unclean %>%
  mutate(
    Month = ymd(Month),
    Gender = as.factor(Gender),
    Married = as.factor(Married),
    ApplicantIncome = as.numeric(str_remove_all(ApplicantIncome, "\\D")),
    Loan_Status = as.factor(Loan_Status)
  )
df_clean %>% head(n = 10)
```

```
## # A tibble: 10 x 7
##    Month      Loan_ID  Gender Married ApplicantIncome Credit_History Loan_Status
##    <date>     <chr>    <fct>  <fct>             <dbl>          <dbl> <fct>
##  1 2020-01-01 LP001002 Male   No                 5849              1 Y
##  2 2020-01-02 LP001003 Male   Yes                4583              1 N
##  3 2020-01-03 LP001005 Male   Yes                3000              1 Y
##  4 2020-01-04 LP001006 Male   Yes                2583              1 Y
##  5 2020-01-05 LP001008 Male   No                 6000              1 Y
##  6 2020-01-06 LP001011 Male   Yes                5417              1 Y
##  7 2020-01-07 LP001013 Male   Yes                2333              1 Y
##  8 2020-01-08 LP001014 Male   Yes                3036              0 N
##  9 2020-01-09 LP001018 Male   Yes                4006              1 Y
## 10 2020-01-10 LP001020 Male   Yes               12841              1 N
```

Subsection 4

Data aggregation

Counting data

Now that you know how to transform your data, you may want to know more about aggregating your data to make it more interpretable. So far, going back to the home loan applicants data set, we have been working at the same level as the initial data, where every observation corresponds to one applicant. In this chapter, you will learn to take many observations (applicants) and summarize them into one. This technique is making datasets manageable and interpretable.

The simple way to aggregate data is to count it, i.e. to find out the number of observations, by using the `count()` function from the `dplyr` package. Counting is very useful type of aggregation when you are starting to analyze your data.

```
df %>% count()
```

```
## # A tibble: 1 x 1
##       n
##   <int>
## 1   614
```

This tells us there are 614 observations in the table. You do not get much information from total number of observations. The real value of the function is when you use a specific variable to count.

```
df %>% count(Gender, sort = TRUE)
```

```
## # A tibble: 3 x 2
##   Gender     n
##   <chr>  <int>
## 1 Male     489
## 2 Female   112
## 3 <NA>      13
```

Here we see how many female and male applicants there are. We also see that there are 13 missing values. If you want to sort counted data to find most common observations, the `count()` function provides an optional second argument `sort`, which sorts counted data if set to `TRUE`.

Instead of finding number of applicants by gender, assume we want to know the total value of their income. We can add additional argument `wt`, which means that `n` column will be weighted by the `ApplicantIncome` columns.

```
df %>% count(Gender, wt = ApplicantIncome, sort = TRUE)
```

```
## # A tibble: 3 x 2
##   Gender       n
##   <chr>    <dbl>
## 1 Male   2663319
## 2 Female  520069
## 3 <NA>    134336
```

You may want to count combinations of two columns, for example `Gender` and `Married` columns.

```
df %>% count(Gender, Married)
```

```
## # A tibble: 8 x 3
##   Gender Married     n
##   <chr>  <chr>   <int>
## 1 Female No         80
## 2 Female Yes        31
## 3 Female <NA>        1
## 4 Male   No        130
## 5 Male   Yes       357
## 6 Male   <NA>        2
## 7 <NA>   No          3
## 8 <NA>   Yes        10
```

Group by and summarize

Count is just a special case of more general set of `dplyr` functions: `group by()` and `summarize()`. With the `summarize()` function, many observations are aggregated into one observation by using different summary functions such as:

- `sum()`
- `mean()`
- `median()`
- `min()`
- `max()`
- `n()`

For example, to find the total value of applicants income, you use the `summarize()` function and the `sum()` function inside it.

```
df %>% summarize(total_income = sum(ApplicantIncome))
```

```
## # A tibble: 1 x 1
##   total_income
##          <dbl>
## 1      3317724
```

Instead of aggregating the entire table, we can aggregate data within groups. Let's find the minimum loan amount and average applicants' income by gender. To do this, we firstly need to group data by the `Gender` column by using the `group by()` function. Keep in mind that you can define multiple variables inside a `summarize()` function, and aggregate each of them in different way.

```
df %>%
  group_by(Gender) %>%
  summarize(
    min_amount = min(LoanAmount, na.rm = TRUE), #remove missing values
    avg_income = mean(ApplicantIncome)
  )
```

```
## `summarise()` ungrouping output (override with `.groups` argument)
```

```
## # A tibble: 3 x 3
##   Gender min_amount avg_income
##   <chr>       <dbl>      <dbl>
## 1 Female          9      4643.
## 2 Male           17      5446.
## 3 <NA>           94     10334.
```

Also, you can group by multiple columns at the same time. For example, we can group by both `Gender` and `Education` column and calculate the maximum value of loan amount. This will result in one row for each combination of these two categorical variables.

```
df %>%
  group_by(Gender, Education) %>%
  summarize(
    max_amount = max(LoanAmount, na.rm = TRUE)
  )
```

```
## # A tibble: 6 x 3
## # Groups:   Gender [3]
##   Gender Education    max_amount
##   <chr>  <chr>             <dbl>
## 1 Female Graduate            600
## 2 Female Not Graduate        225
## 3 Male   Graduate            650
## 4 Male   Not Graduate        279
## 5 <NA>   Graduate            700
## 6 <NA>   Not Graduate        112
```

Be aware that obtained results after grouping data by multiple columns are still grouped by the first variable. This is useful if you want to continue doing additional aggregations. However, in order to turn off grouping, you need to use `ungroup()` function at the end of the expression.

```
df %>%
  group_by(Gender, Education) %>%
  summarize(
    max_amount = max(LoanAmount, na.rm = TRUE)
  ) %>%
  ungroup()
```

```
## # A tibble: 6 x 3
##   Gender Education    max_amount
##   <chr>  <chr>             <dbl>
## 1 Female Graduate            600
## 2 Female Not Graduate        225
## 3 Male   Graduate            650
## 4 Male   Not Graduate        279
## 5 <NA>   Graduate            700
## 6 <NA>   Not Graduate        112
```

Subsection 5

Joining data

The information you need is not always confined in one table. Sometimes you have to join multiple tables together, so that they can be analyzed in combination. In this subsection we will learn four mutating joins: inner join, left join, right join, and full join, and two filtering joins: semi join and anti join.

Mutating joins

Take a look at two tables we will work with. These are extracts from the table we are already familiar with.

```
(joins_1 <- read_csv("../data/joins_1.csv"))
```

```
## # A tibble: 5 x 3
##   Loan_ID  Education    Income_ID
##   <chr>    <chr>            <dbl>
## 1 LP001005 Graduate            54
## 2 LP001006 Not Graduate        98
## 3 LP001008 Graduate            65
## 4 LP001011 Graduate            88
## 5 LP001013 Not Graduate        16
```

```
(joins_2 <- read_csv("../data/joins_2.csv"))
```

```
## # A tibble: 5 x 3
##      ID Education         ApplicantIncome
##   <dbl> <chr>                       <dbl>
## 1    65 Master's degree              6000
## 2    88 Bachelor's degree            5417
## 3    16 High school                  2333
## 4    80 High school                  3036
## 5    11 Bachelor's degree            4006
```

The first table contains information about applicants' education and income id. In the second table we have the id column, again education and applicants' income. We have to recognize common id that appears in both tables, which are `Income_ID` and `ID` columns from the first and second table respectively. Assume we want to observe only applicants with available information both about their education and income, i.e. we are interested in applicants that appear in both tables. We would use the `inner_join()` function.

```
joins_1 %>%
  inner_join(joins_2, by = c("Income_ID" = "ID"))
```

```
## # A tibble: 3 x 5
##   Loan_ID  Education.x  Income_ID Education.y       ApplicantIncome
##   <chr>    <chr>            <dbl> <chr>                       <dbl>
## 1 LP001008 Graduate            65 Master's degree              6000
## 2 LP001011 Graduate            88 Bachelor's degree            5417
## 3 LP001013 Not Graduate        16 High school                  2333
```

Notice the argument **by** which tells inner join how to match the tables, by linking `Income_ID` from the first table with `ID` column from the second table. When the variables you are joining by have the same name, you can just write `by = "common_id"`.

From the output table we see that information from both tables are combined and that there are three clients that appear in both tables. Since both tables contain a variable called `Education`, output table includes both columns with `.x` and `.y` suffixes, because you can not have two columns with the same name. You can customize this to be more readable and intuitive by using the additional argument `suffix`.

```
joins_1 %>%
  inner_join(
    joins_2, by = c("Income_ID" = "ID"),
    suffix = c("_joins_1", "_joins_2")
  )
```

```
## # A tibble: 3 x 5
##   Loan_ID  Education_joins_1 Income_ID Education_joins_2 ApplicantIncome
##   <chr>    <chr>                  <dbl> <chr>                      <dbl>
## 1 LP001008 Graduate                  65 Master's degree             6000
## 2 LP001011 Graduate                  88 Bachelor's degree           5417
## 3 LP001013 Not Graduate              16 High school                 2333
```

After joining joins_1 to joins_2 table we obtain table with exactly one observation for each applicant that appears in both tables. That is because in the joins_2 table each applicant has only one source of income. Assume now we have applicants that have multiple sources of income as in table joins_3.

```
(joins_3 <- read_csv("../data/joins_3.csv"))
```

```
## # A tibble: 6 x 3
##      ID Education         ApplicantIncome
##   <dbl> <chr>                      <dbl>
## 1    65 Master's degree             6000
## 2    88 Bachelor's degree           5417
## 3    16 High school                 2333
## 4    80 High school                 3036
## 5    11 Bachelor's degree           4006
## 6    65 Master's degree             7500
```

Applicant with ID = 65 has two observations in joins_3 table.

What would happen if we join joins_1 and joins_3 table?

```
joins_1 %>%
  inner_join(
    joins_3, by = c("Income_ID" = "ID"),
    suffix = c("_joins_1", "_joins_3")
  )
```

```
## # A tibble: 4 x 5
##   Loan_ID  Education_joins_1 Income_ID Education_joins_3 ApplicantIncome
##   <chr>    <chr>                <dbl> <chr>                       <dbl>
## 1 LP001008 Graduate                65 Master's degree              6000
## 2 LP001008 Graduate                65 Master's degree              7500
## 3 LP001011 Graduate                88 Bachelor's degree            5417
## 4 LP001013 Not Graduate            16 High school                  2333
```

As you see, we obtain table with four observations. Applicant with two sources of income now appears twice in the joined table. Information from the first table (joins_1) for this applicant duplicates. Paying a close attention to the number of rows before and after joining tables is an important part of understanding your data.

We saw that `inner_join()` keeps only observations that appear in both tables. But, what if we want to keep only applicants from the first table when we are joining tables. In that case, we would use the `left_join()` function.

```
joins_1 %>%
  left_join(
    joins_2, by = c("Income_ID" = "ID"),
    suffix = c("_joins_1", "_joins_2")
  )
```

```
## # A tibble: 5 x 5
##   Loan_ID  Education_joins_1 Income_ID Education_joins_2 ApplicantIncome
##   <chr>    <chr>                <dbl> <chr>                       <dbl>
## 1 LP001005 Graduate                54 <NA>                           NA
## 2 LP001006 Not Graduate            98 <NA>                           NA
## 3 LP001008 Graduate                65 Master's degree              6000
## 4 LP001011 Graduate                88 Bachelor's degree            5417
## 5 LP001013 Not Graduate            16 High school                  2333
```

Notice that columns from the `joins_2` table have missing `NA` values for applicants that do not appear in the `joins_2` table. We did not have these rows when we used `inner_join` because first two applicants are not part of the second table. Here, we keep all the observations from the first ("left") table, no matter if they exist in the second ("right") table.

Similarly, there is also the `right_join()` function. Just as `left_join` keep all the observations from the "left" table, whether or not they appear in the "right" table, a `right_join` keeps all the observations from the "right" table, whether or not they appear in the "left" table. Practically the opposite.

```
joins_1 %>%
  right_join(
    joins_2, by = c("Income_ID" = "ID"),
    suffix = c("_joins_1", "_joins_2")
  )
```

```
## # A tibble: 5 x 5
##   Loan_ID  Education_joins_1 Income_ID Education_joins_2 ApplicantIncome
##   <chr>    <chr>                <dbl> <chr>                      <dbl>
## 1 LP001008 Graduate                65 Master's degree             6000
## 2 LP001011 Graduate                88 Bachelor's degree           5417
## 3 LP001013 Not Graduate            16 High school                 2333
## 4 <NA>     <NA>                    80 High school                 3036
## 5 <NA>     <NA>                    11 Bachelor's degree           4006
```

Now, missing values appear in the columns from the first table for the applicants that appear only in the second table.

Instead of keeping all observations from the left or right tables, you might want to keep all observations from both tables, whether or not they match to each other. The `full_join()` function keeps all the observations from both left and right tables.

```
joins_1 %>%
  full_join(
    joins_2, by = c("Income_ID" = "ID"),
    suffix = c("_joins_1", "_joins_2")
  )
```

```
## # A tibble: 7 x 5
##   Loan_ID  Education_joins_1 Income_ID Education_joins_2 ApplicantIncome
##   <chr>    <chr>                <dbl> <chr>                       <dbl>
## 1 LP001005 Graduate                54 <NA>                           NA
## 2 LP001006 Not Graduate            98 <NA>                           NA
## 3 LP001008 Graduate                65 Master's degree              6000
## 4 LP001011 Graduate                88 Bachelor's degree            5417
## 5 LP001013 Not Graduate            16 High school                  2333
## 6 <NA>     <NA>                    80 High school                  3036
## 7 <NA>     <NA>                    11 Bachelor's degree            4006
```

We see that all applicants are in the joined table, even ones that appear only in one of the tables.

Filtering joins

So far we learned about mutating joins which combine the columns from two tables. Filtering joins, on the other hand, keep or remove observations from the first table, but do not add new variables.

The `semi_join()` function filters only applicants from the first table that appear in the second table. The final table contains only information from the first table. Therefore, we do not need to specify the `suffix` argument.

```
joins_1 %>%
  semi_join(joins_2, by = c("Income_ID" = "ID"))
```

```
## # A tibble: 3 x 3
##   Loan_ID  Education    Income_ID
##   <chr>    <chr>            <dbl>
## 1 LP001008 Graduate            65
## 2 LP001011 Graduate            88
## 3 LP001013 Not Graduate        16
```

The opposite of a `semi_join` is an `anti_join`. It filters applicants from the first table that do not appear in the second table.

```
joins_1 %>%
  anti_join(joins_2, by = c("Income_ID" = "ID"))
```

```
## # A tibble: 2 x 3
##   Loan_ID  Education    Income_ID
##   <chr>    <chr>            <dbl>
## 1 LP001005 Graduate            54
## 2 LP001006 Not Graduate        98
```

Subsection 6

Saving results

When we finish our analysis, we might want to save our intermediate or final results. Before, we saw how to import data using `utils` and `readr` R packages. Analogously to `read.csv()` and `read.table()` functions from the `utils` package used for importing data, there are `write.csv()` and `write.table()` functions used for exporting data to `csv` or `txt` format. Notice that `write.delim` function does not exist, so you need to specify the `sep` argument inside the `write.table()` function to define the separating parameter.

Again, `readr` provides similar functions which work almost twice as fast as functions from the `utils` package. In order to save results in `csv` format you use `write_csv()` function, whereas `write_tsv()` is used for saving results in `txt` format with tab separation. This package provides `write_delim()` function if you want to specify delimiter on your own with the `delim` argument.

```
write_csv(file_csv, "../data/file_name.csv")
write_tsv(file_tsv, "../data/file_name.txt")
write_delim(file_slash, "../data/file_name.txt", delim = "/")
```

It is possible to save (write) a single R object to a specified format with the extension `.rds`. This is R's own file format and it has multiple advantages to some other storage approaches. Firstly, it is faster to restore the data to R and secondly it keeps R specific information encoded in the data (e.g., attributes, variable types, etc). `saveRDS()` and `readRDS()` functions are used for saving and restoring single R objects. However, the `readr` package provides its own functions for working with `RDS` files, such as `write_rds()` and `read_rds()`.

```
write_rds(file_rds, "../data/object.RDS")
file_name <- read_rds("../data/object.RDS")
```

Subsection 7

Data Visualization in R

Data visualization is an essential skill for data scientists. It combines statistics and design, and emphasizes accurate representation and interpretation of data. Although R does provide built-in plotting functions, the `ggplot2` package will allow you to make nearly any kind of data visualization, customized to your specific needs. This package contains functions that are able to provide beatiful plots of your data.

In order to create a plot, you firstly call the `ggplot()` function which creates a blank canvas. With aesthetic mappings you specify how to map variables to visual aspects. Afterwards, you add geometric objects ("geoms") which will show up on the plot. Finally, you add optional arguments (titles, axes labels, themes...). All these iterative steps can be connected with the + sign. This iterative nature of `ggplot2` is one of the things that makes it so powerful. Since the `ggplot2` is part of the `tidyverse` group of packages, your dataframe can be connected with the `ggplot()` function using pipes.

Geometries

The `ggplot2` package supports a number of different types of geoms, including:

- `geom_point` for drawing individual points (scatter plots),
- `geom_line` for drawing lines (line charts),
- `geom_bar` and `geom_col` for drawing bars (bar charts),
- `geom_histogram` for drawing histograms,
- `geom_smooth` for drawing smoothed lines,
- `geom_polygon` for drawing arbitrary shapes. . .

Let's explore some of these shapes in the following examples. We will use both Facebook stock and loan applicants datasets.

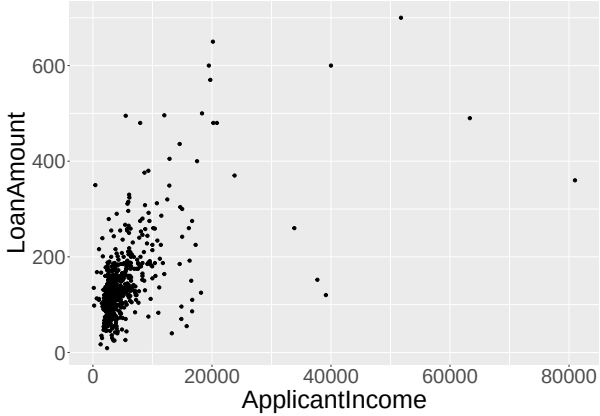
```r
library(ggplot2)

df %>% head(n = 6)
```

```
## # A tibble: 6 x 13
##   Loan_ID Gender Married Dependents Education Self_Employed ApplicantIncome
##   <chr>   <chr>  <chr>   <chr>      <chr>     <chr>                   <dbl>
## 1 LP0010~ Male   No      0          Graduate  No                       5849
## 2 LP0010~ Male   Yes     1          Graduate  No                       4583
## 3 LP0010~ Male   Yes     0          Graduate  Yes                      3000
## 4 LP0010~ Male   Yes     0          Not Grad~ No                       2583
## 5 LP0010~ Male   No      0          Graduate  No                       6000
## 6 LP0010~ Male   Yes     2          Graduate  Yes                      5417
## # ... with 6 more variables: CoapplicantIncome <dbl>, LoanAmount <dbl>,
## #   Loan_Amount_Term <dbl>, Credit_History <dbl>, Property_Area <chr>,
## #   Loan_Status <chr>
```

```r
fb <- read_csv("../data/FB.csv")
fb %>% head(n = 6)
```

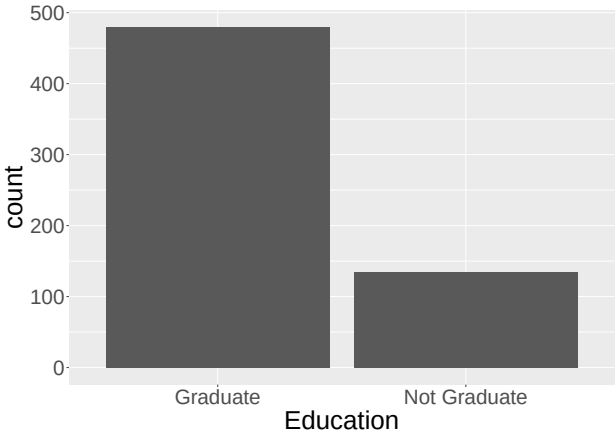
```
## # A tibble: 6 x 7
##   Index      FB.Open FB.High FB.Low FB.Close FB.Volume FB.Adjusted
##   <date>       <dbl>   <dbl>  <dbl>    <dbl>     <dbl>       <dbl>
## 1 2013-01-02    27.4    28.2   27.4     28    69846400        28
## 2 2013-01-03    27.9    28.5   27.6     27.8  63140600        27.8
## 3 2013-01-04    28.0    28.9   27.8     28.8  72715400        28.8
## 4 2013-01-07    28.7    29.8   28.6     29.4  83781800        29.4
## 5 2013-01-08    29.5    29.6   28.9     29.1  45871300        29.1
## 6 2013-01-09    29.7    30.6   29.5     30.6 104787700        30.6
```

```r
df %>%
  ggplot(aes(
    x = ApplicantIncome, y = LoanAmount
  )) +
  geom_point()
```

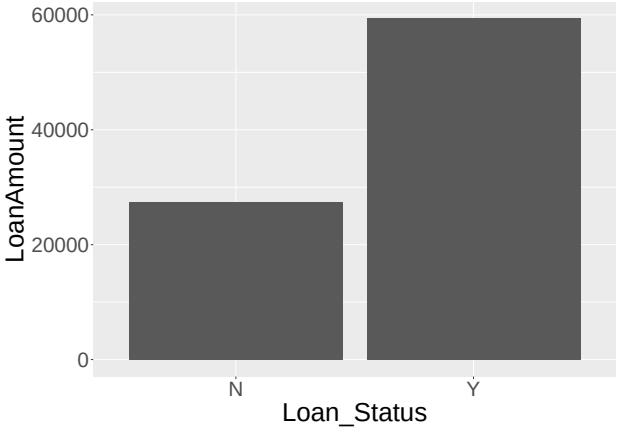



```r
fb %>%
  ggplot(aes(
    x = Index, y = FB.Adjusted
  )) +
  geom_line()
```

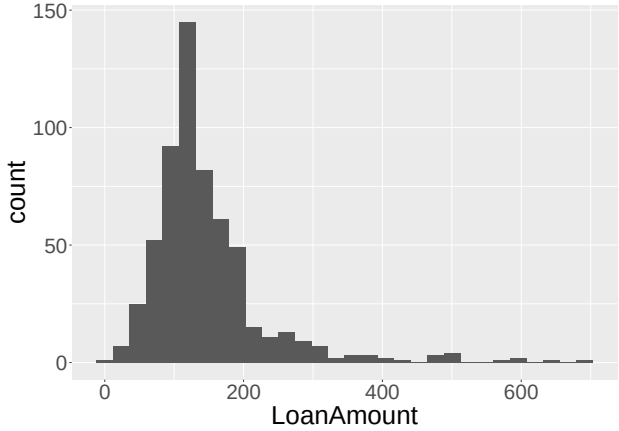
```r
df %>%
  ggplot(aes(Education)) +
  geom_bar()
```




```r
df %>%
  ggplot(aes(Loan_Status, LoanAmount)
  geom_col()
```



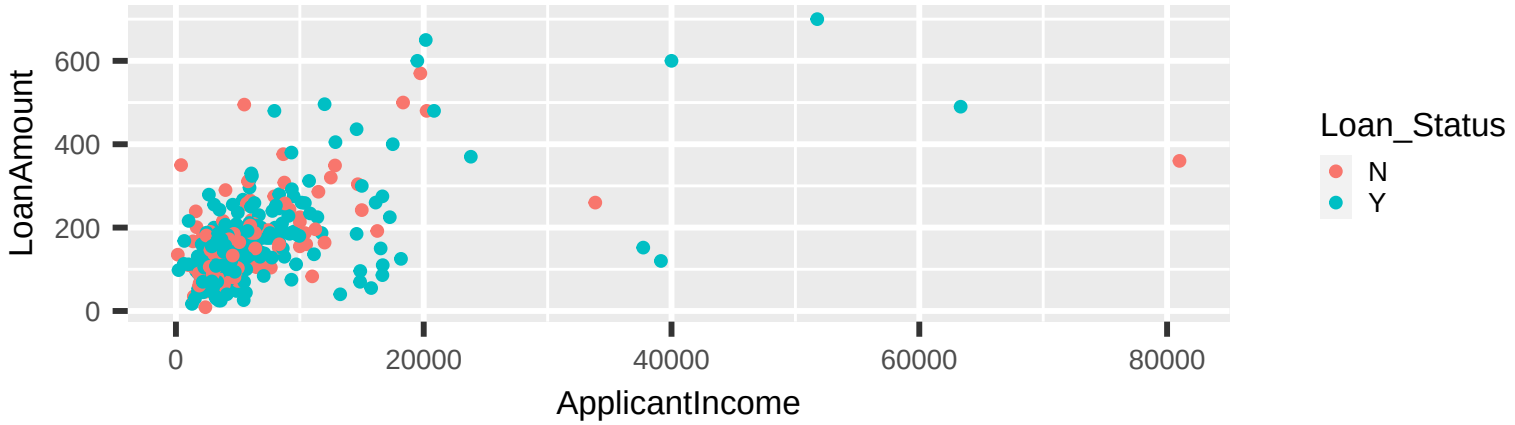

```r
df %>%
  ggplot(aes(LoanAmount)) +
  geom_histogram()
```



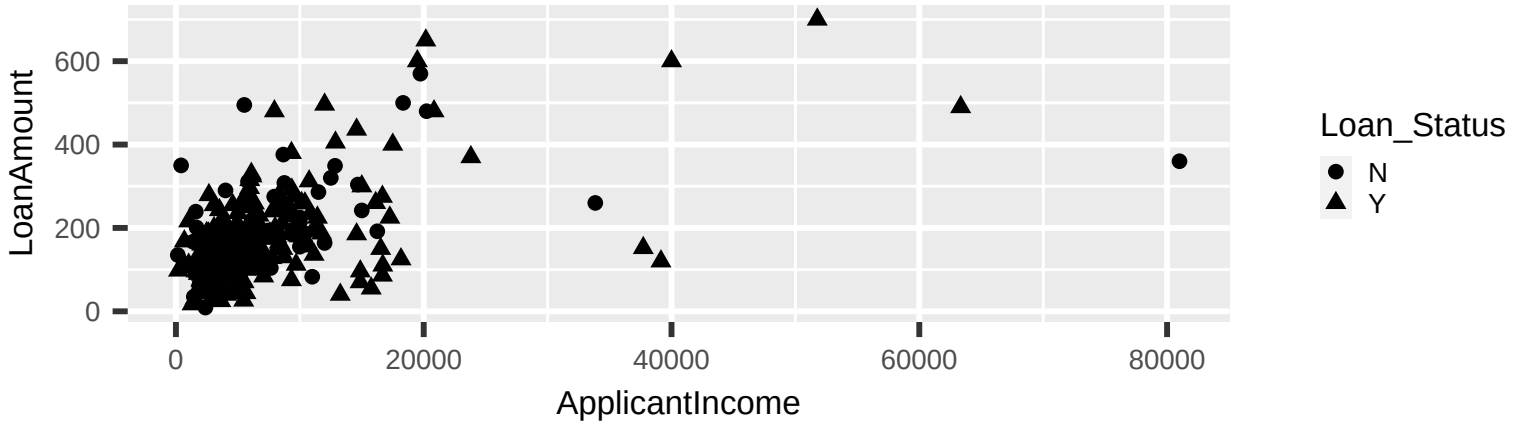
Aesthetics

You can convey information about your data by mapping the aesthetics in your plot to the variables in your dataset. An aesthetic is a visual property of the objects in your plot which includes things like the size, the shape, or the color of your points, lines, or bars.

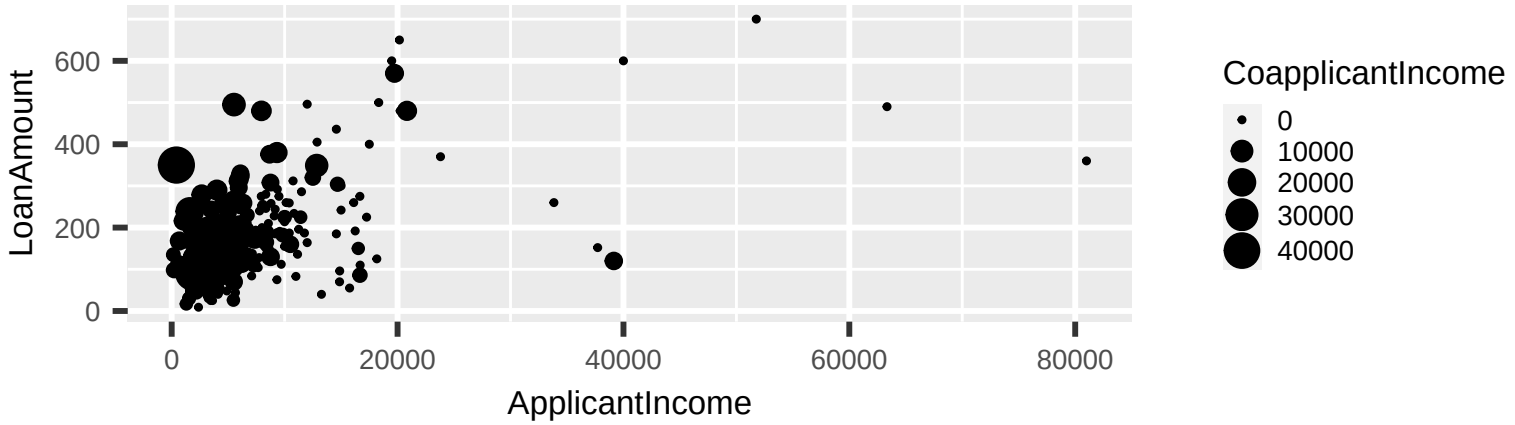
```r
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point()
```



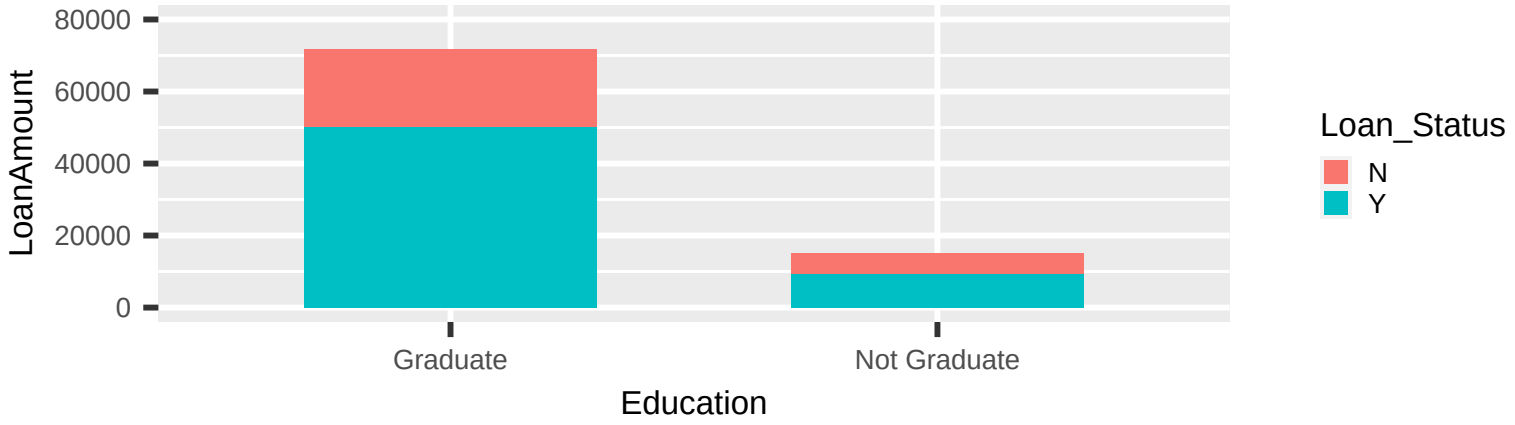

```r
df %>%
  ggplot(aes(ApplicantIncome, LoanAmount, shape = Loan_Status)) +
  geom_point()
```



```
df %>%
  ggplot(aes(ApplicantIncome, LoanAmount, size = CoapplicantIncome)) +
  geom_point()
```

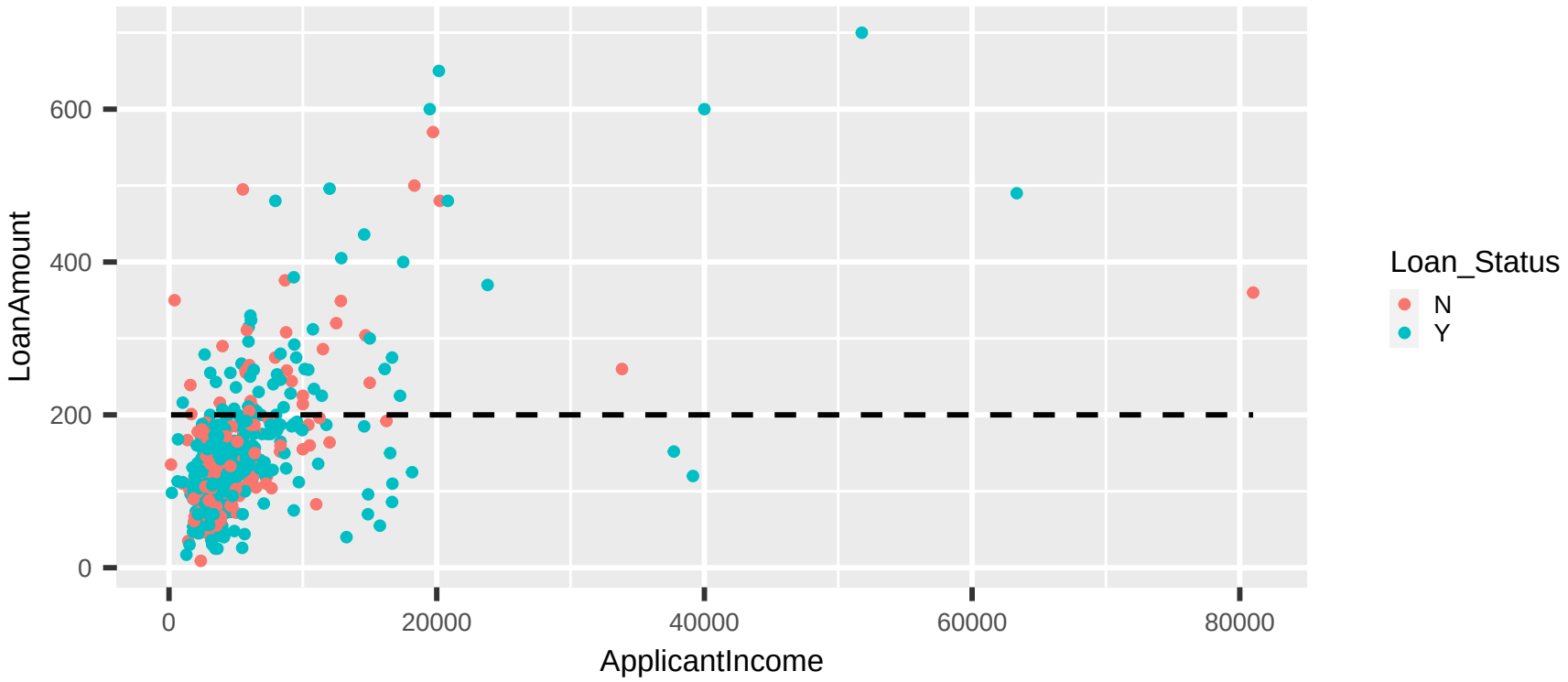



```
df %>%
  ggplot(aes(Education, LoanAmount, fill = Loan_Status)) +
  geom_col()
```



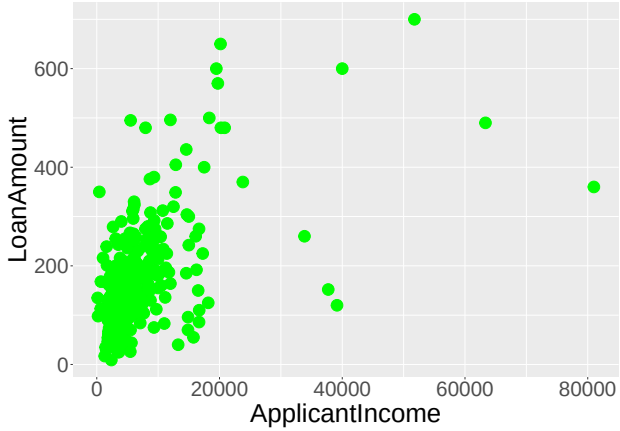
We call aesthetics in the `aes` function. However, we could also called aesthetics in the `geom` layer. It provides the same results. This is typically done if we do not want all layers have the same aesthetics or we are using different data sources.

```
df %>%
  ggplot() +
  geom_point(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_line(aes(x = ApplicantIncome, y = 200), linetype = "dashed")
```

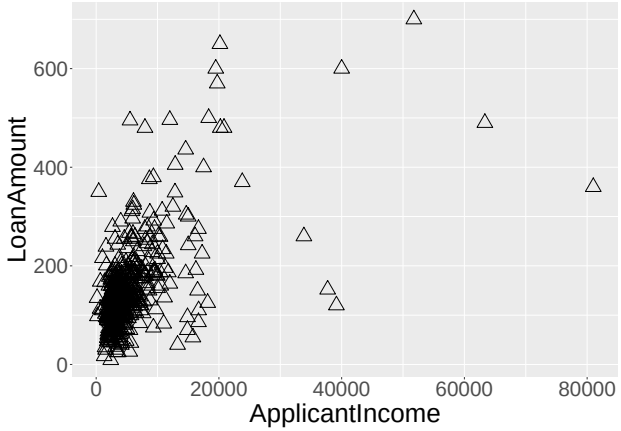


All of previously mentioned aesthetics also exist as attributes. Attributes are always called in the geom layer. For example, if we want to change color of points in previous scatter plot to green, we set the color argument outside the aes function in the geom layer.

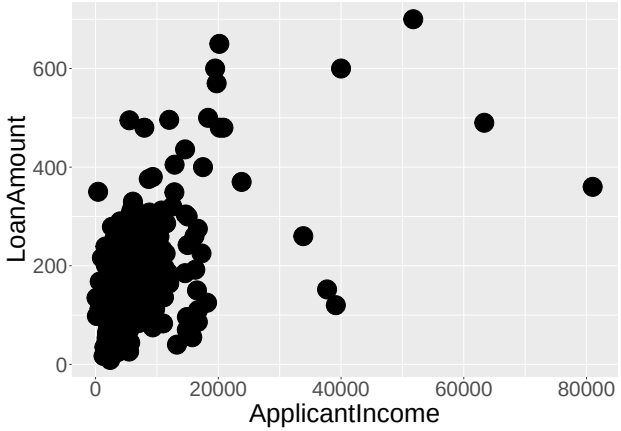
```
df %>%
  ggplot(aes(
    ApplicantIncome, LoanAmount
  )) +
  geom_point(color = "green")
```




```
df %>%
  ggplot(aes(
    ApplicantIncome, LoanAmount
  )) +
  geom_point(shape = 2)
```




```
df %>%
  ggplot(aes(
    ApplicantIncome, LoanAmount
  )) +
  geom_point(size = 6)
```

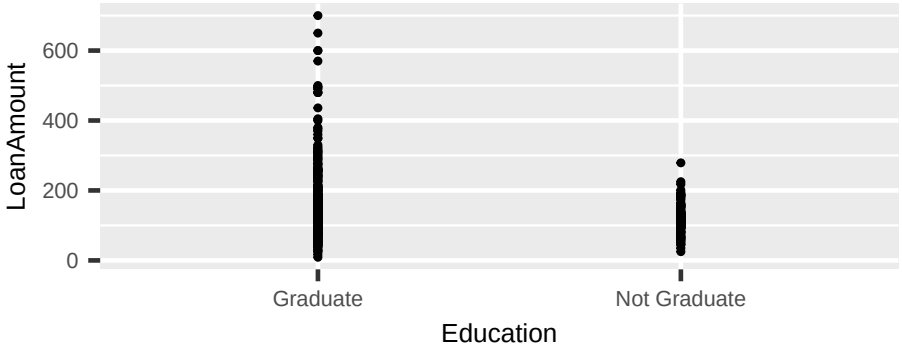


Let's now explore how to modify aesthetics. A common adjustment is position. It specifies the adjustment of overlapping bar or points on a single layer. There are a lot of possibilities such as `identity`, `dodge`, `stack`, `jitter`... The `identity` position is set by default. It means that the value will be positioned in the plot exactly by the value from the data frame. We see that there are many points that overlap on our scatter plot and it is hard to distinguish them. To solve this, we need to add some random noise on both axes to see regions of high density. This refers to `jittering`. `jitter` can be used as argument inside the `geom` layer, but each position can also be accessed as a function.

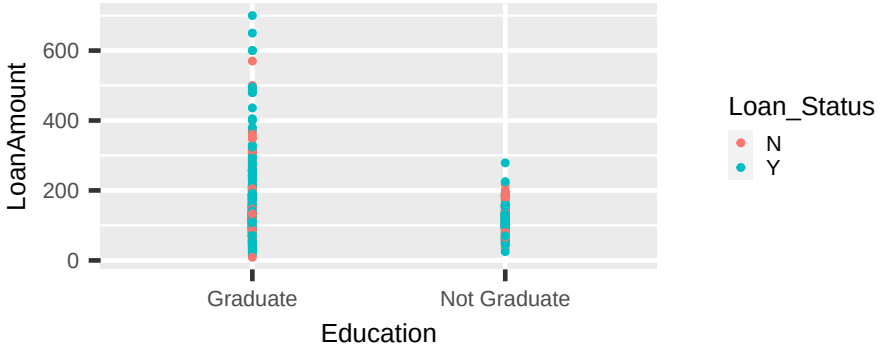
```
position = position_jitter(width = NULL, height = NULL, seed = NA)
```

Specifying position as a function is convenient as you can set specific arguments for the position such as width, which defines how much noise should be added.

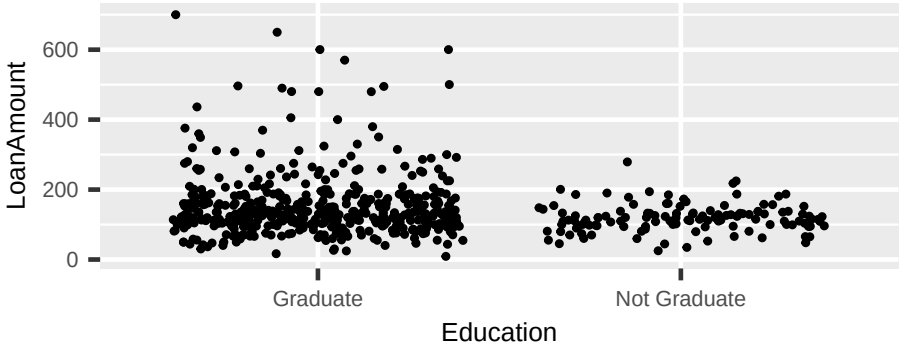
```r
df %>%
  ggplot(aes(
    x = Education, y = LoanAmount
  )) +
  geom_point()
```



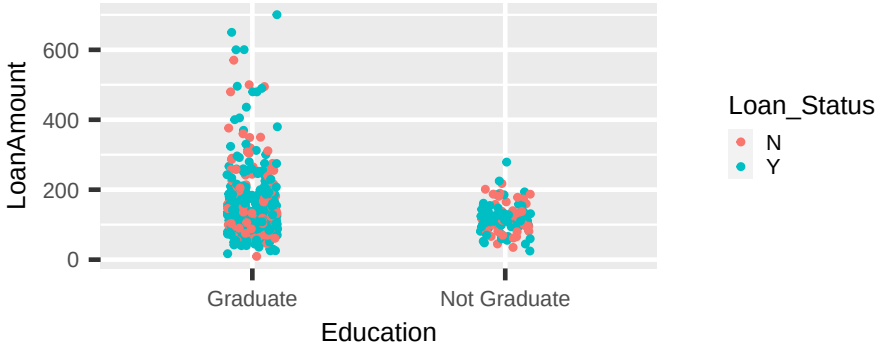

```r
df %>%
  ggplot(aes(
    Education, LoanAmount, col = Loan_Status
  )) +
  geom_point()
```




```r
df %>%
  ggplot(aes(
    x = Education, y = LoanAmount
  )) +
  geom_point(position = "jitter")
```

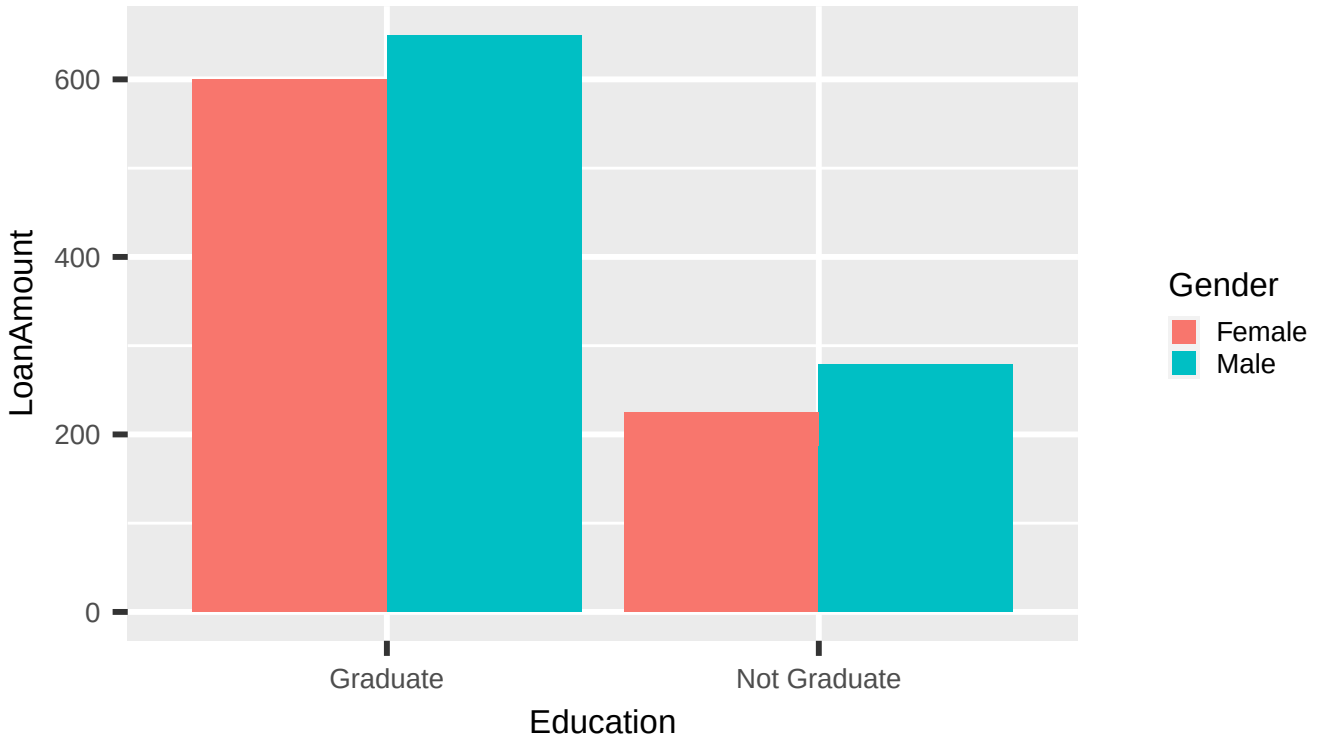



```r
df %>%
  ggplot(aes(
    Education, LoanAmount, col = Loan_Status
  )) +
  geom_point(position = position_jitter(width = 0.1))
```



Another popular position function is dodge. It preserves the vertical position of an geom while adjusting the horizontal position[8].

```
df %>%
  filter(!is.na(Gender)) %>%
  ggplot(aes(Education, LoanAmount, fill = Gender)) +
  geom_col(position = "dodge")
```

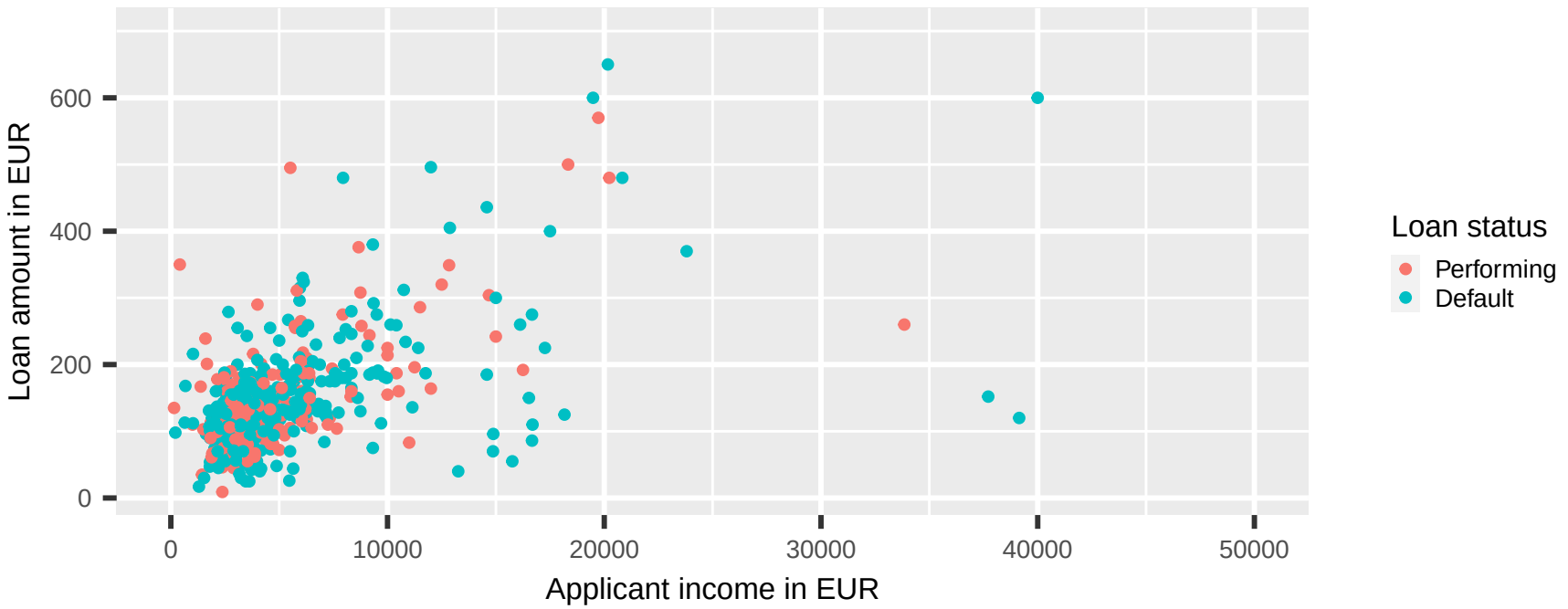


[8] Missing values are omitted for a better representation.

Each of the aesthetics is a scale which we map data onto. `x` and `y` are scales, and `color`, `shape` or `fill` are also scales. We can access all the scales with the scale functions, such as `scale_x_continuous`, `scale_y_continuous`, `scale_color_discrete`, `scale_fill_discrete`... As you see, the second part of the scale function construction defines which scale we want to modify. The third part matches the type of data we are using, where `discrete` means we are working with categorical data, while `continuous` is used when the data is numerical.

The first argument of the scale function is the name of the scale. There are many other arguments. Apart from the `name` argument, `limits`, `breaks` and `labels` are the most common ones. To define the scale's range, we use `limits`. With the `breaks` argument, we control the tick mark positions, while the `labels` adjusts the category names.

```r
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  scale_x_continuous(
    name = "Applicant income in EUR",
    limits = c(0, 50000),
    breaks = seq(0, 50000, 10000)
  ) +
  scale_y_continuous(name = "Loan amount in EUR") +
  scale_color_discrete(
    name = "Loan status",
    labels = c("Performing", "Default")
  )
```

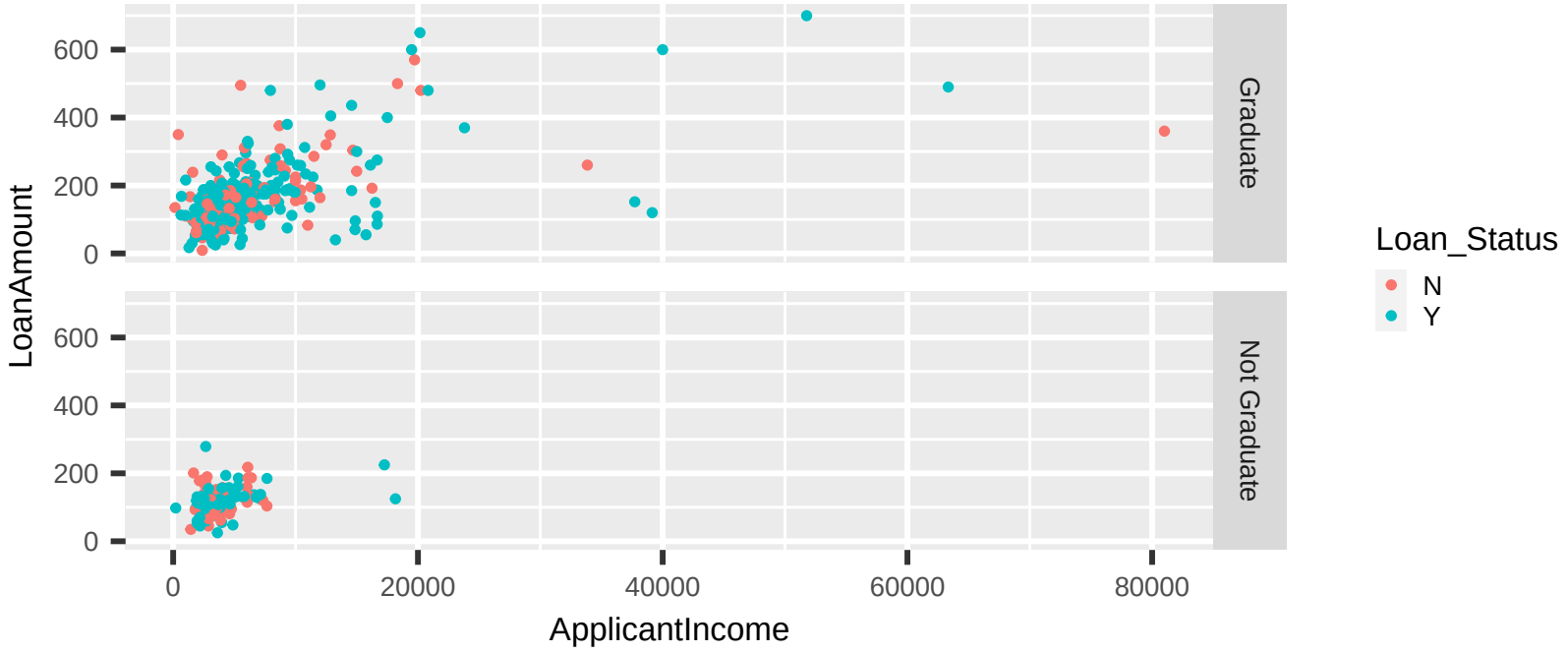


Facets

Facets are very useful tool in data visualization. The idea is that we can split up a large, complex plot to generate smaller plots with the exact same coordinate system. On each plot we present different information and we can compare it more easily. So the primary use of facets is to add another categorical variable to a plot, but also to aid in visual perception.

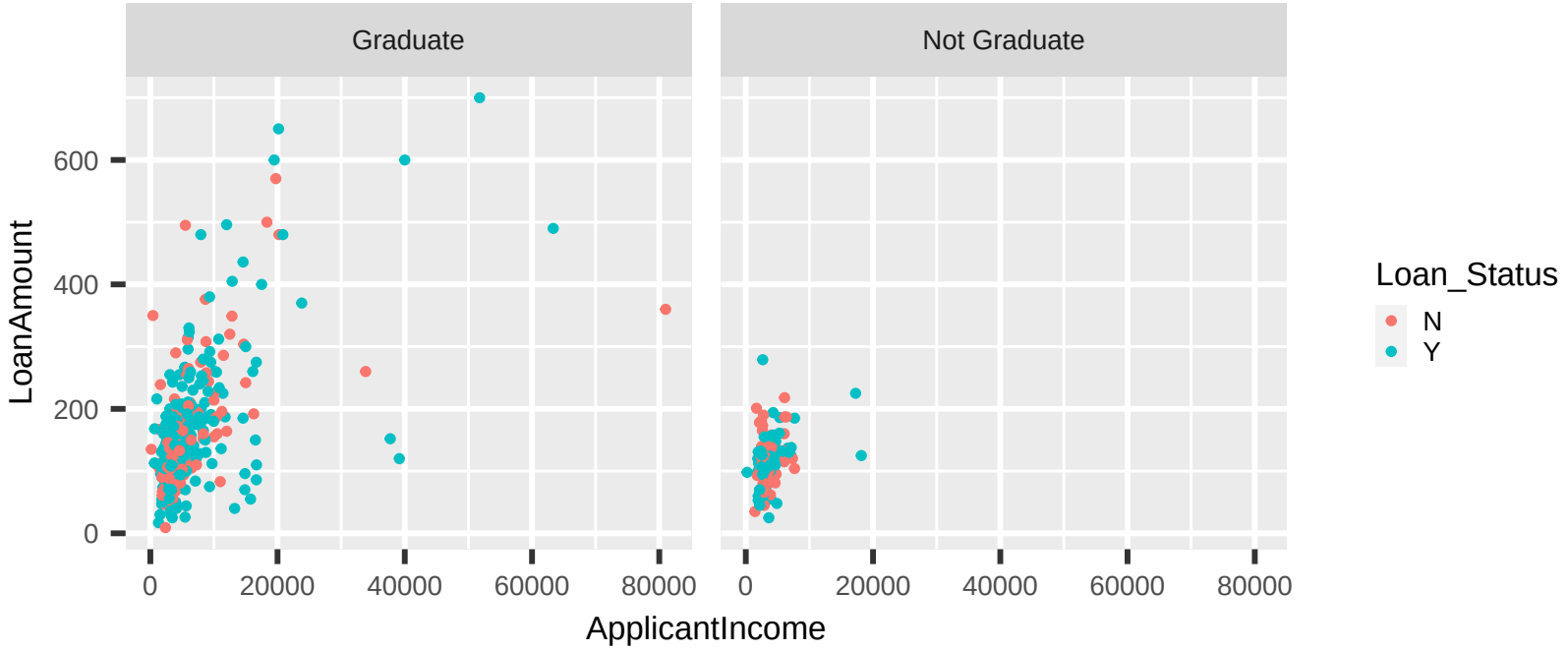
Have a look again at our scatter plot. With the `color` aesthetics we separated loan status of applicants. We can add a `facet_grid` layer to add another variable to a plot, for example `Education`. The `cols` argument splits plots into columns, whereas the `rows` argument splits them by rows according to the variable specified in the `vars` function.

```
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_grid(rows = vars(Education))
```



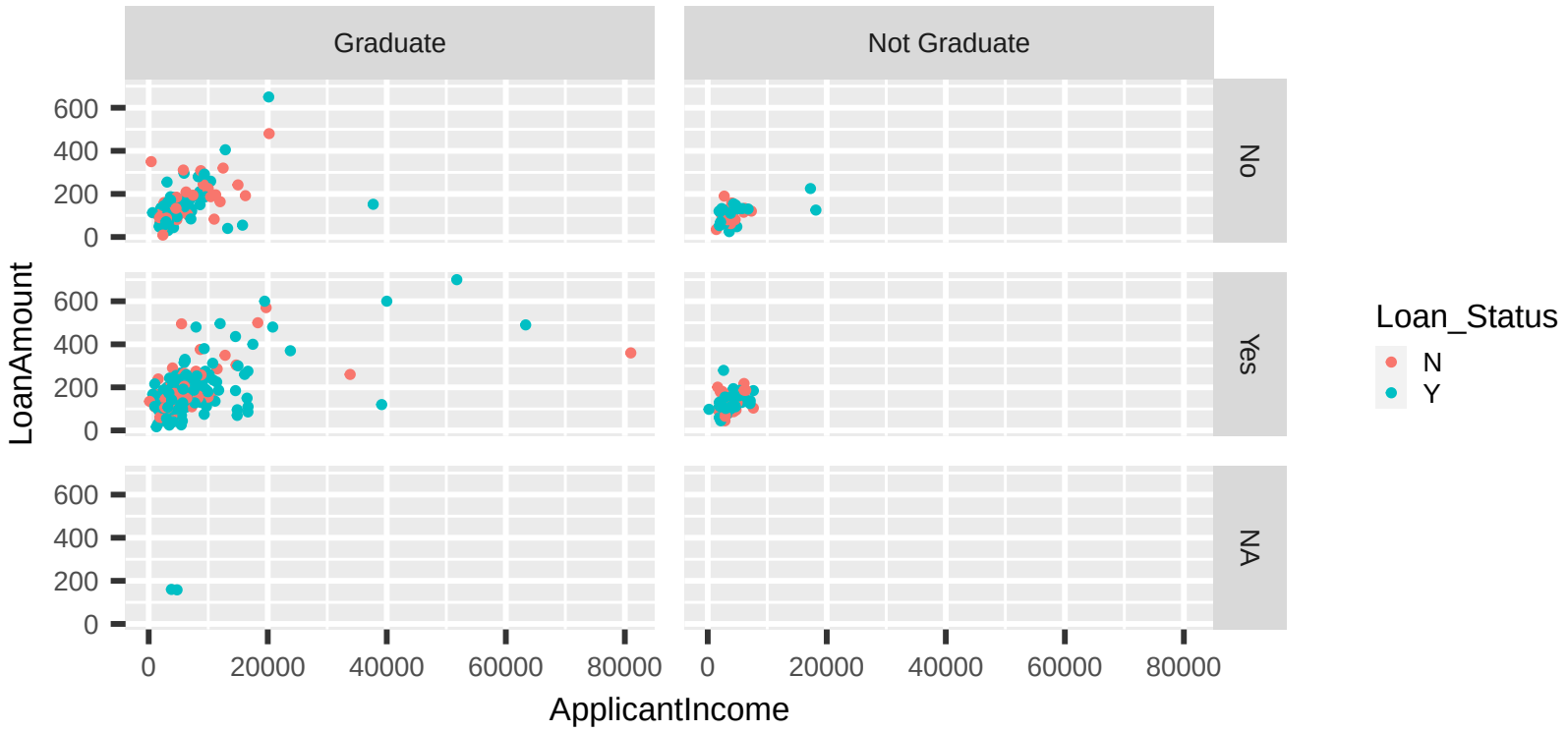
Instead of using these arguments, the same thing can be achieved by using the ~ notation. Everything on the left of the ~ will split according to rows, and everything on the right will be split according to columns.

```
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_grid(~Education)
```



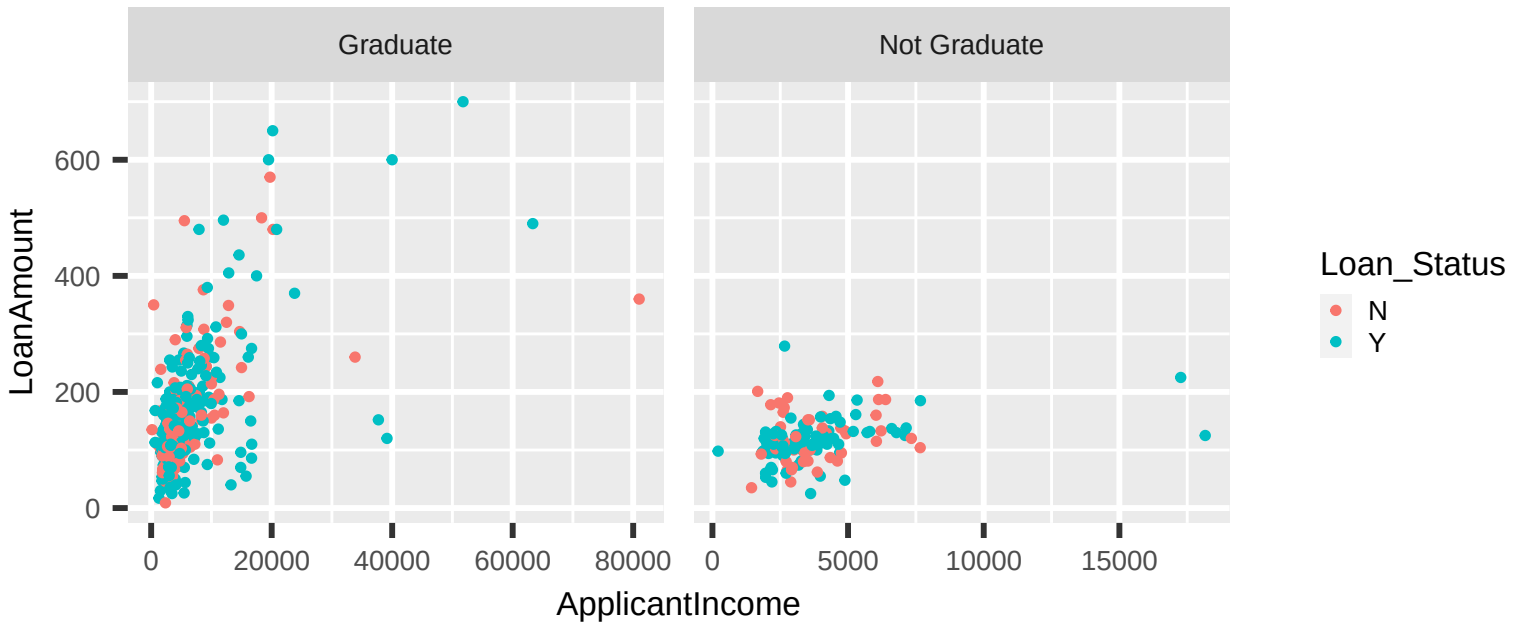
Of course, you can also split according to both columns and rows by using two or multiple different variables.

```
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_grid(Married ~ Education)
```



We can adjust plotting space by using the `scales` argument. If we set this argument to `free_x`, we allow each variable to have their own x axis. Similarly, we can do the same thing for the y axis by setting the `scales` argument to `free_y`. In order to adjust both axes, we set the `scales` argument to `free`.

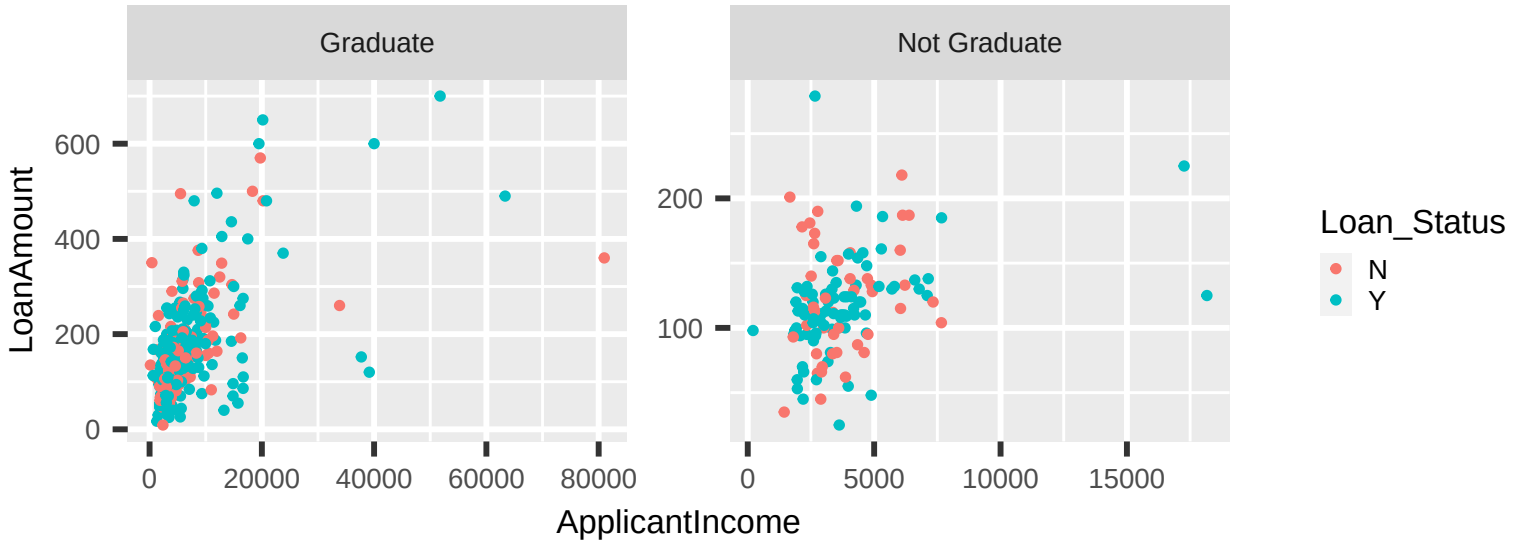
```
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_grid(~Education, scales = "free")
```



By using the `facet_grid` layer we see that only one axis can be adjusted based on the way we are splitting the plots. If the split it by column as in previous example, only `x` axis is adjusted with the `scales` function.

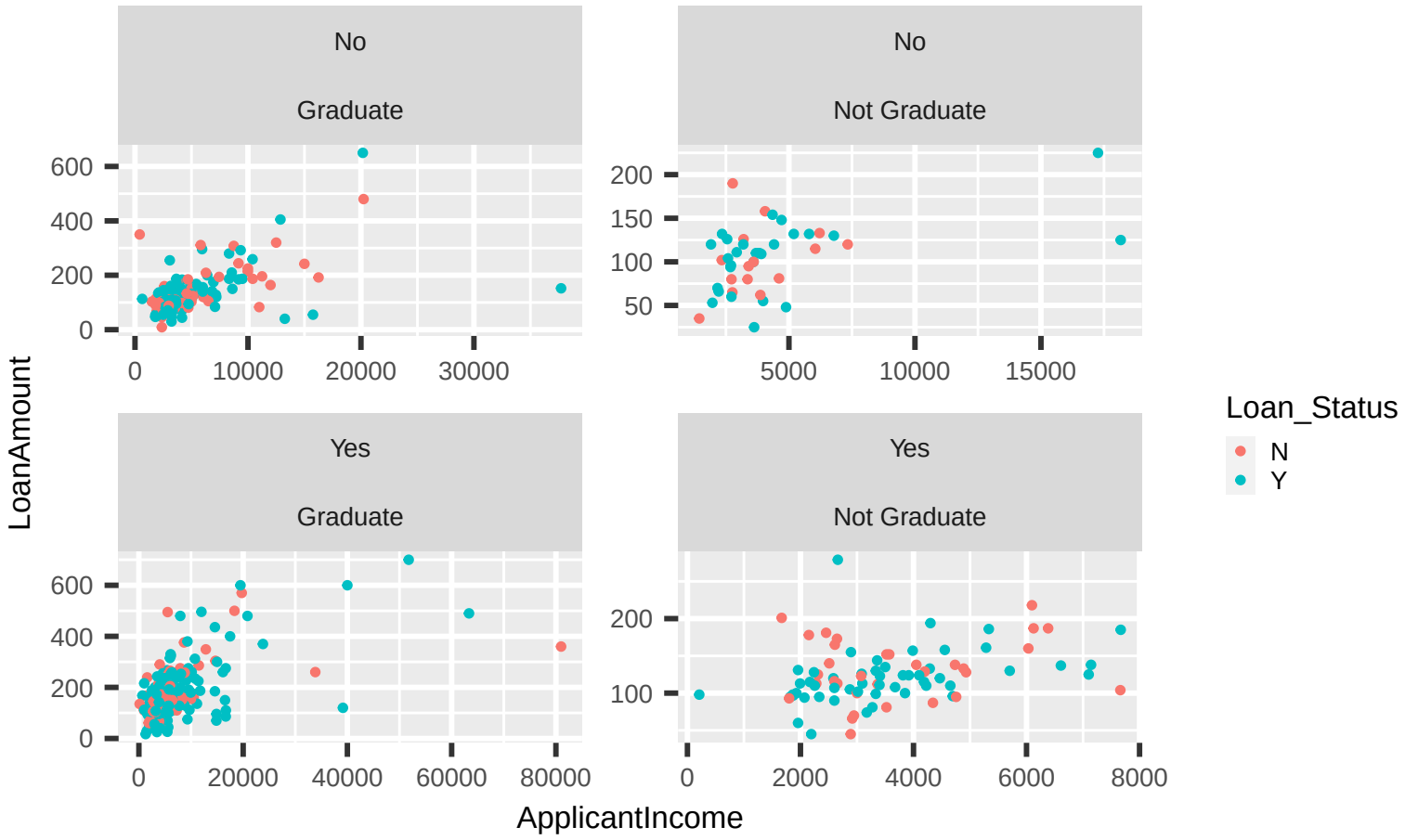
However, what if we want to allow each column to have it's own `x` and `y` axes dependent on their data. This can be achieved with the `facet_wrap` layer.

```
df %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_wrap(~Education, scales = "free")
```



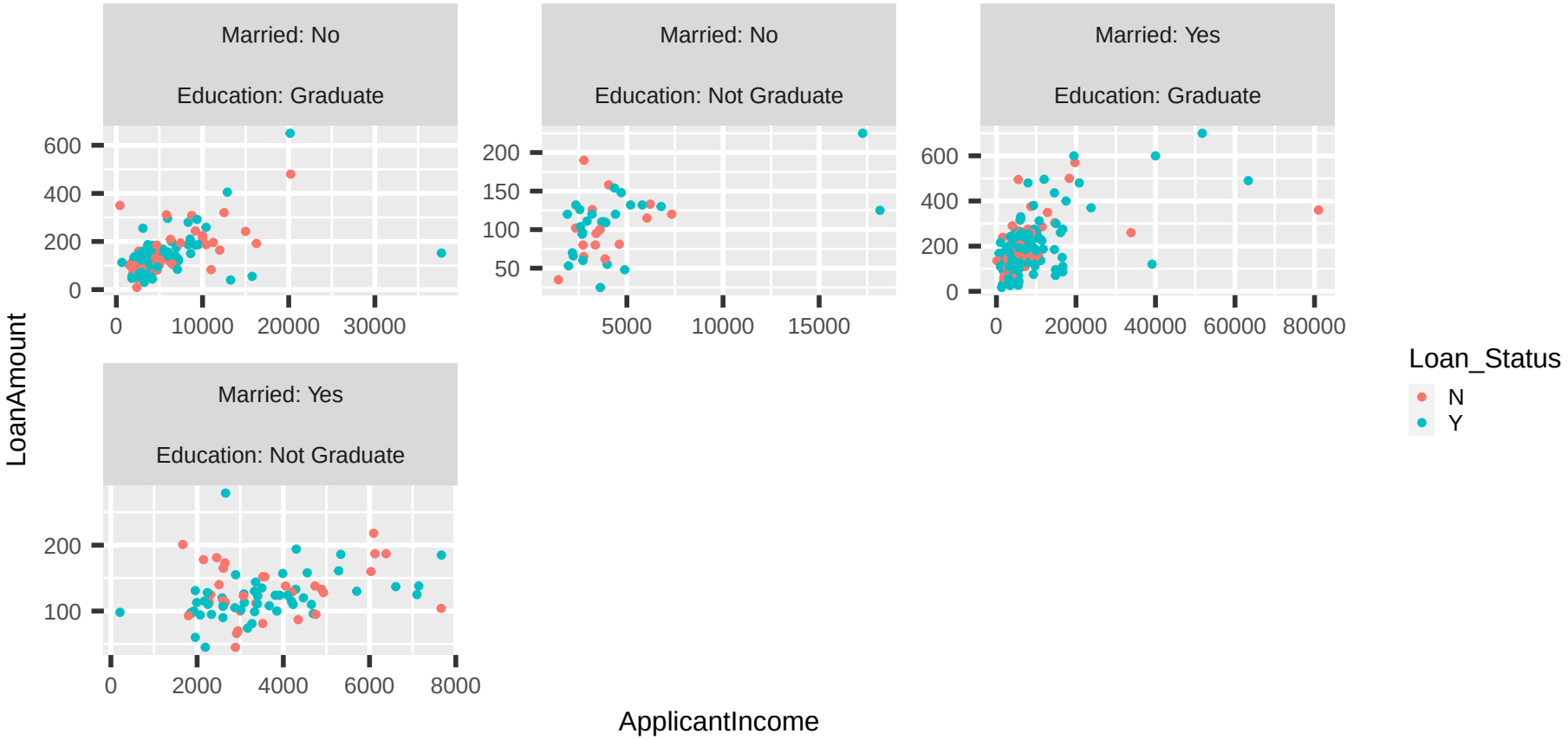
Here is one more example of the `facet_wrap` function in case we split data both by columns and rows.

```
df %>%
  filter(!is.na(Married)) %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_wrap(Married ~ Education, scales = "free")
```



Other useful arguments are `ncol` and `nrow` for specifying number of rows and columns, and `labeller` which is used for formatting the strip labels of facet grids and wraps.

```
df %>%
  filter(!is.na(Married)) %>%
  ggplot(aes(x = ApplicantIncome, y = LoanAmount, color = Loan_Status)) +
  geom_point() +
  facet_wrap(Married ~ Education, scales = "free", ncol = 3, labeller = label_both)
```

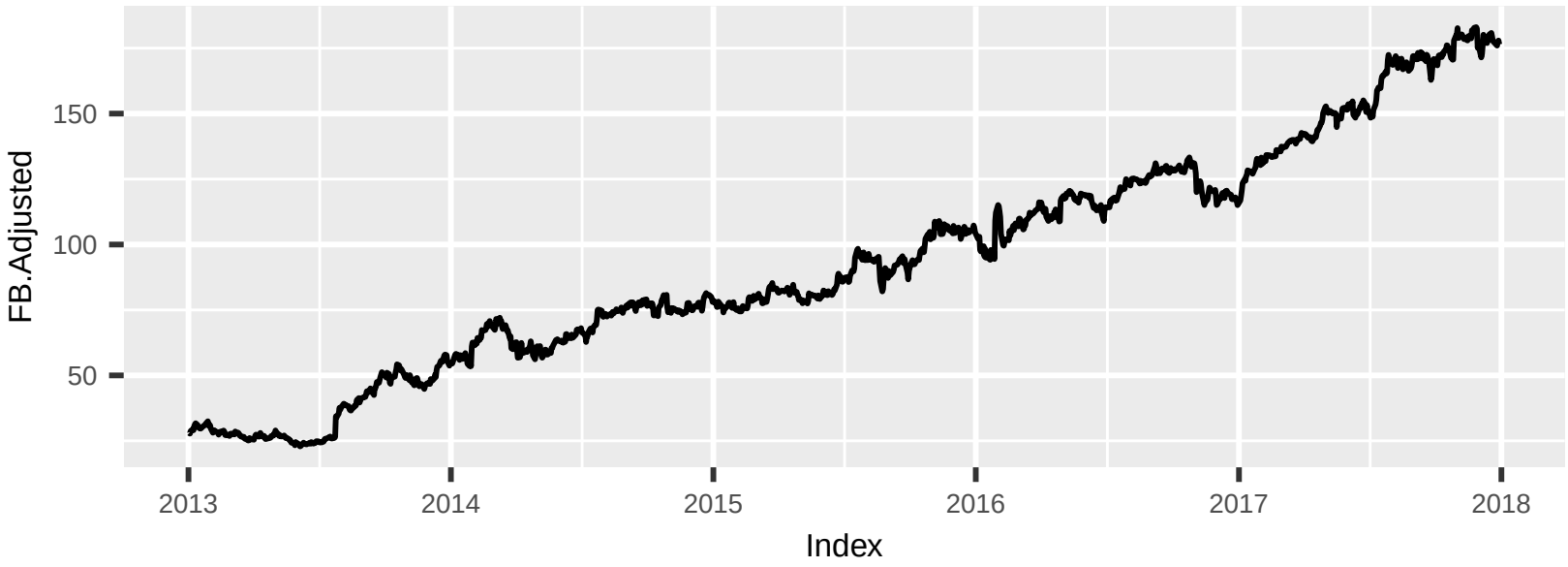


Subsection 8

Time Series

A time series is a sequence of data in chronological order. More precisely, time series are series of data points listed at successive equally spaced points in time. We find time series everywhere, especially in Financial data, such as daily log returns on a stock, monthly values of Consumer Price Index (CPI) or GDP and many other.

```
fb %>%
  ggplot(aes(Index, FB.Adjusted)) +
  geom_line() +
  scale_x_date(
    date_breaks = "1 year",
    date_labels = "%Y"
  )
```



ts objects

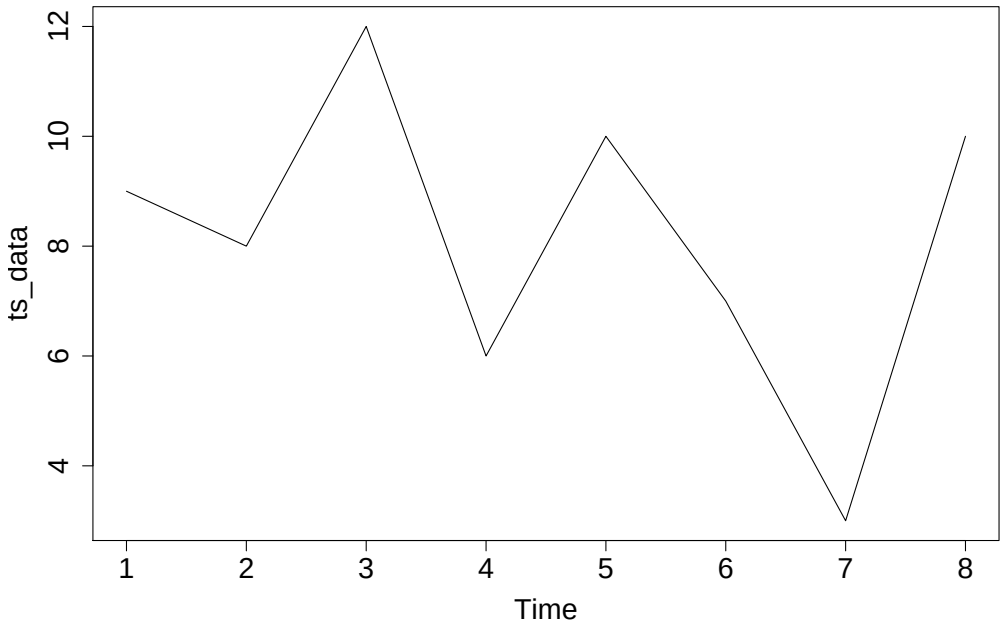
Time series not only include vector of numbers but also time indices for each observation. In order to create a time series object, you can apply the `ts()` function. It is more convenient to work with time series presented as `ts` objects as there are many methods and functions available that utilize time series attributes. Consider some vector of numbers that represent your time series data.

```
vctr <- c(9, 8, 12, 6, 10, 7, 3, 10)
ts_data <- ts(vctr)
ts_data
```

```
## Time Series:
## Start = 1
## End = 8
## Frequency = 1
## [1]  9  8 12  6 10  7  3 10
```

With the `plot()` function you can graphically present `ts` objects. When we plot the result, the time index and label are added on the horizontal axis. By default, R uses observation index as time index starting from 1.

```
plot(ts_data)
```

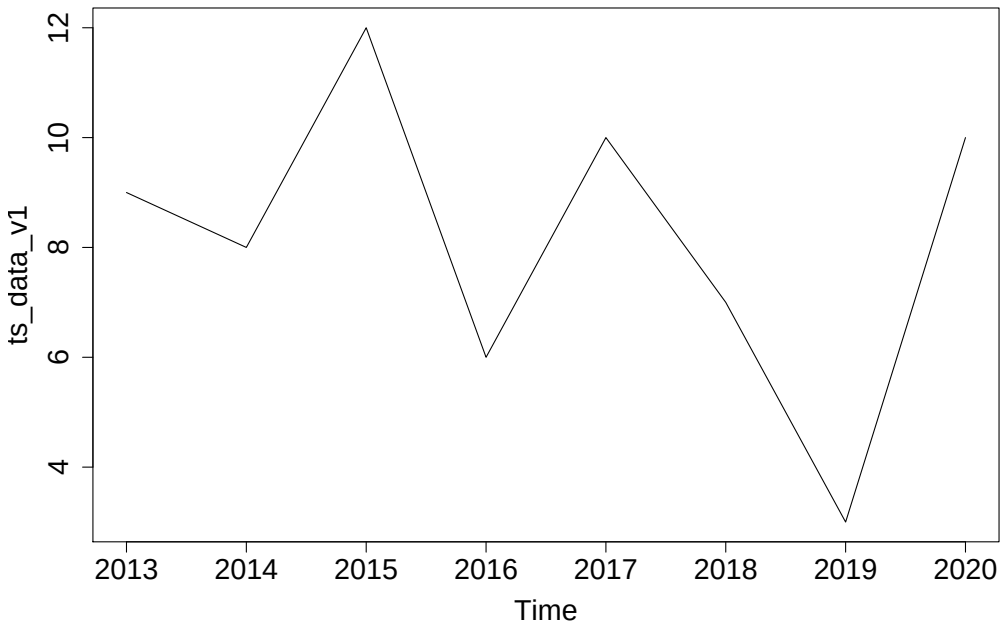


However, if we want our time series to start in some specific year with one observation per year, we need to add additional arguments of the `ts()` function, `start` and `frequency`.

```
ts_data_v1 <- ts(vctr, start = 2013, frequency = 1)
ts_data_v1
```

```
## Time Series:
## Start = 2013
## End = 2020
## Frequency = 1
## [1]  9  8 12  6 10  7  3 10
```

```
plot(ts_data_v1)
```



xts and zoo objects

In this chapter we will introduce xts and zoo objects and their components. Compared to ts, zoo objects can be irregularly spaced and can use most common index classes. Rather than calculating the index by using start, end and frequency arguments, zoo stores the index item by item or row by row.

xts objects are an extension of zoo objects designed to make using time series easy. They are subclass of zoo objects and consist of data that is represented as matrix, and a vector of times which correspond to each row. It is best to think of xts objects as normal R matrices, but with more possibilities. In order to work with xts and zoo objects, we need to install and load the xts and zoo packages.

```
library(xts)
library(zoo)
```

Now, let's create a simple matrix that will represent our data.

```
ts_data <- matrix((1:6)^3, ncol = 1)
ts_data
```

```
##      [,1]
## [1,]    1
## [2,]    8
## [3,]   27
## [4,]   64
## [5,]  125
## [6,]  216
```

Each row is an observation of time, so we need to define a vector of dates. This index need to be a true time object, not a string or number.

```
time_vector <- ymd("2020-01-01") + months(0:5)
time_vector
```

```
## [1] "2020-01-01" "2020-02-01" "2020-03-01" "2020-04-01" "2020-05-01"
## [6] "2020-06-01"
```

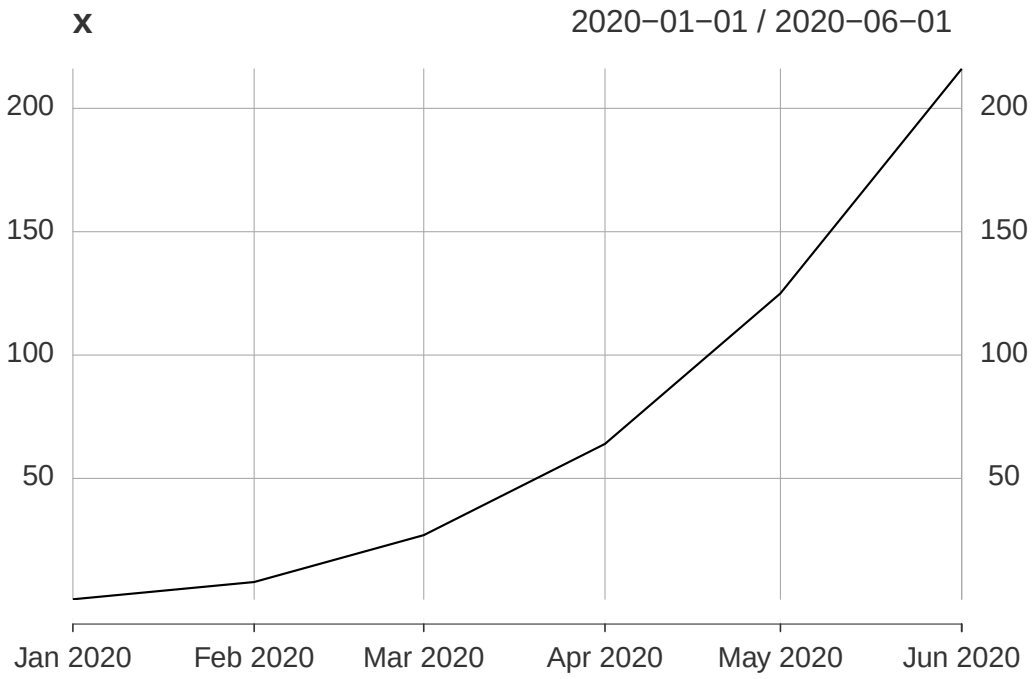
To create `xts` object we need to join matrix of data with the vector of dates by calling the `xts()` function. First argument `x` is our time series data saved in a matrix, whereas the second argument is `order.by` where we put our vector of dates. `xts()` function will reorder your time vector if it is not in increasing order of time, to ensure you have a properly ordered time series.

```
x <- xts(x = ts_data, order.by = time_vector)
x
```

```
##            [,1]
## 2020-01-01    1
## 2020-02-01    8
## 2020-03-01   27
## 2020-04-01   64
## 2020-05-01  125
## 2020-06-01  216
```

Again, you can plot `xts` objects using the `plot` function.

```
plot(x)
```



In order to extract the data from the xts object, we use the `coredata()` and the `index()` functions from the zoo package. The `coredate()` function allows you to get your matrix back, while with the `index()` you extract the time vector.

```r
coredata(x)
```

```
##      [,1]
## [1,]    1
## [2,]    8
## [3,]   27
## [4,]   64
## [5,]  125
## [6,]  216
```

```r
index(x)
```

```
## [1] "2020-01-01" "2020-02-01" "2020-03-01" "2020-04-01" "2020-05-01"
## [6] "2020-06-01"
```

Handling missing values

When working with time series, you often have to find a way to handle missing values. The `zoo` package includes various functions which enable tackling missing values using different strategies. One of the most common ways to deal with `NAs` is to fill them with some specific replacement values by using the `na.fill()` function.

Also, you can remove all rows with missing values with the `na.omit()` function, or remove only rows with `NA` values at the beginning or end of the series with the `na.trim()` function.

However, you usually do not want to have gaps in your data. Therefore you may wish to interpolate values between observations using the `na.approx()` function based on the distance between time stamps.

Finally, maybe the most convenient way to handle missing values is to apply so called *last observation carried forward*, or *locf*. `na.locf()` function from the `zoo` package takes the last known non-NA value, and moves it forward. You can also apply it in reverse order, setting the `fromLast` argument to `TRUE`.

```
ts_na <- xts(
  x = matrix(c(1, 2, NA, 4, NA, 7), ncol = 1),
  order.by = ymd("2020-01-01") + 0:5
)

cbind(
  ts_na = ts_na,
  ts_fill = na.fill(ts_na, "-9999"),
  approx = na.approx(ts_na),
  locf = na.locf(ts_na),
  locf_rev = na.locf(ts_na, fromLast = T)
)
```

```
##            ts_na ts_fill approx locf locf_rev
## 2020-01-01     1       1    1.0    1        1
## 2020-01-02     2       2    2.0    2        2
## 2020-01-03    NA   -9999    3.0    2        4
## 2020-01-04     4       4    4.0    4        4
## 2020-01-05    NA   -9999    5.5    4        7
## 2020-01-06     7       7    7.0    7        7
```

Lags and differences

Explaining and calculating lags and differences for our data is the last thing we will cover in this chapter. In time series analysis, you will often need to compare values from different periods. We perform this with the `lag()` function[9]. The first argument of the `lag()` function is the `xts` time series, and number of lags is defined with the `k` argument. The positive `k` value will shift observations forward in time, whereas the negative one will shift it back in time[10]. `k` can also be a vector of different values.

```
lag(x, k = c(0, -1, 1, 2))
```

```
##            lag0 lag.1 lag1 lag2
## 2020-01-01    1     8   NA   NA
## 2020-02-01    8    27    1   NA
## 2020-03-01   27    64    8    1
## 2020-04-01   64   125   27    8
## 2020-05-01  125   216   64   27
## 2020-06-01  216    NA  125   64
```

[9] The `lag()` function we use here is originally from the `stats` package but is specifically designed for `xts` objects. The function with the same name (`lag()`) can also be found in the `dplyr` package but is not applicable in this context.

[10] Note that this is opposite when working with `ts` objects.

In order to calculate differences between observations in your time series, you use the `diff()` function. We can calculate differences on any lag, and any number of observations. The `lag` argument indicates which lag to use, while the `differences` argument controls the order of the difference.

```r
cbind(
  x = x,
  diff_1 = diff(x, lag = 1),
  diff_2 = diff(x, lag = 2),
  diff_1_2 = diff(x, lag = 1, differences = 2)
)
```

```
##              x diff_1 diff_2 diff_1_2
## 2020-01-01   1     NA     NA       NA
## 2020-02-01   8      7     NA       NA
## 2020-03-01  27     19     26       12
## 2020-04-01  64     37     56       18
## 2020-05-01 125     61     98       24
## 2020-06-01 216     91    152       30
```

Next Steps

RShiny
Machine learning
gganimate
sparklyr
datatable
Git
ggplot2
lubridate
dbplyr
keras
regex
blob
tidyr
stringr
purrr
swirl
flexdashboard
R Markdown